



ALPHA COLLEGE OF ENGINEERING

THIRUMAZHISAI, CHENNAI

(Approved by AICTE and Affiliated to Anna University)

PROJECT REPORT

EventForce Management System

A Salesforce CRM solution for event planning teams

Item	Details
Project Title	EventForce Management System – Salesforce Implementation
Team ID	SWTID-2026-8343
Team Lead	Roobina J (roobina060805@gmail.com)
Team Member	Grace vilma Michael T (gracevilmagracevilma@gmail.com)
Team Member	Dennish P(dennishp50@gmail.com)
Team Member	Vinoth Kumar M(vinothkumar582006@gmail.com)
Team Member	Pradheepa P (pradheepa.p130905@gmail.com)

Alpha College of Engineering, Thirumazhisai, Chennai

Contents

INTRODUCTION

1.1 Project Overview

1.2 Purpose

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

2.2 Empathy Map Canvas

2.3 Brainstorming & Idea Prioritization

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

3.2 Solution Requirements

3.3 Data Flow Diagram

3.4 Technology Stack

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

4.2 Proposed Solution

4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

6. PROJECT DEVELOPMENT PHASE

6.1 Phase 1: Requirement Analysis & Planning

6.2 Phase 2: Backend Development & Configurations

6.3 Phase 3: UI/UX Development & Customization

6.4 Phase 4: Data Migration, Testing & Security

6.5 Phase 5: Deployment, Documentation & Maintenance

7. FUNCTIONAL & PERFORMANCE TESTING

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10 . FUTURE SCOPE

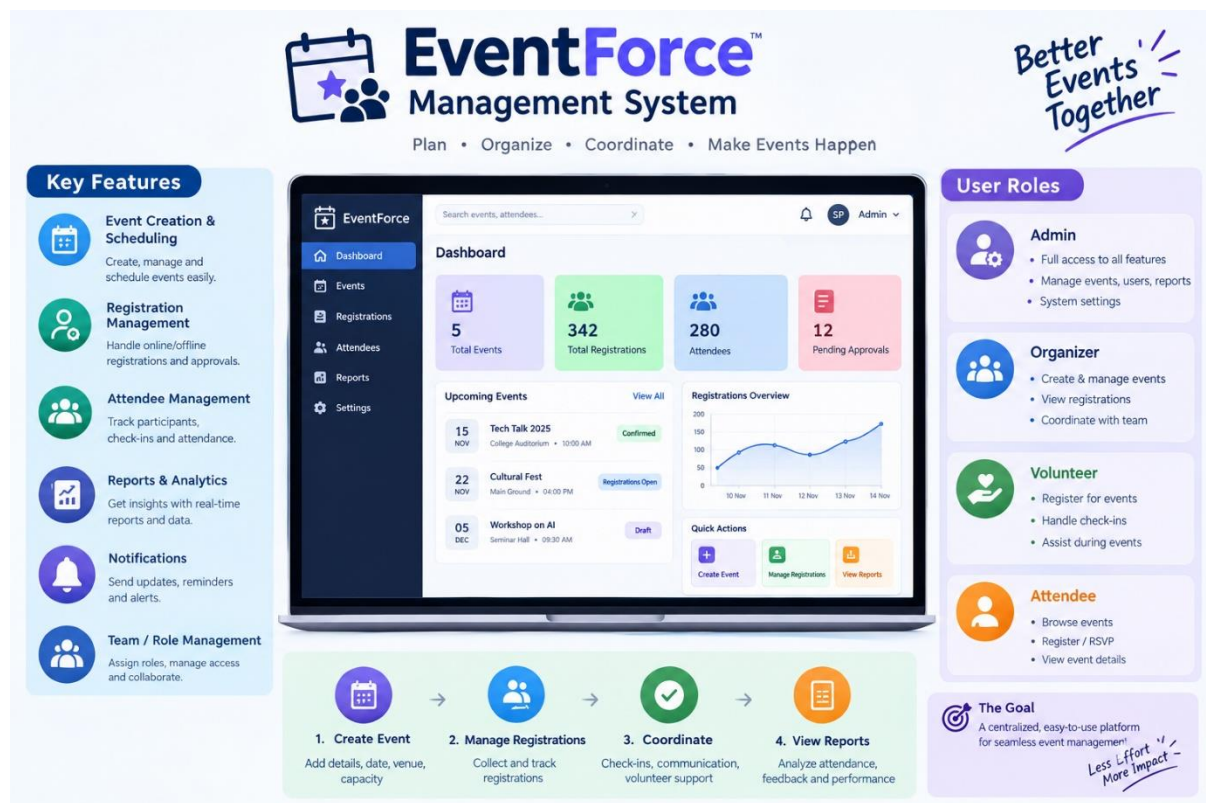
11. APPENDIX

Project Summary

EventForce is a Salesforce-based platform that gives event teams one place to run their work: client bookings, event scheduling, venue reservations, vendor coordination, cancellation approvals, reminders and feedback. Events, Clients, Vendors, Venues and Feedback are held in a structured data model with defined relationships.

Flows send event reminders and keep records current, while an Approval Process governs cancellation requests. Apex classes and triggers add the heavier logic, such as updating venue availability and blocking double bookings. A Lightning App with reports and dashboards shows event schedules, vendor activity and operational performance as they happen.

The result is more efficient event handling, tighter booking coordination, more accurate data, clearer communication, better operational visibility and a better experience for clients.



1. Introduction

1.1 Project Overview

EventForce Management System is a CRM built on Salesforce to bring event planning and day-to-day management onto one platform. It covers clients, events, venues, vendors, feedback and cancellations.

The build relies on custom objects and relationships, validation rules, Flows, an Approval Process, Apex triggers and classes, reports and dashboards. Together these automate the main business steps: managing venue availability, stopping double bookings, sending reminders, approving cancellations, collecting feedback and reporting on operations.

The goal is to cut manual work, make bookings easier to coordinate, keep data accurate, improve communication with clients and internal teams, and give everyone a clear view of event activity.

1.2 Purpose

EventForce exists to give event teams a single, automated Salesforce workspace. Specifically, it is meant to:

- Hold client, event, venue, vendor and feedback information in one place.
- Simplify event booking and venue reservation.
- Use Apex logic to stop venues from being booked twice.
- Automate event reminders and cancellation approvals.
- Improve coordination among clients, coordinators, vendors and venue managers.
- Supply reports and dashboards that support operational decisions.
- Lower manual effort, raise data accuracy and improve the client experience.

Expected Business Value

Business Outcome	Target
Reduction in manual processes	60%
Improvement in booking efficiency	40%
Client satisfaction	Improved through timely reminders and communication
Decision making	Data-driven insight from integrated reports and analytics

2. Ideation Phase

2.1 Problem Statements

Eight customer problem statements were captured, each from the point of view of a specific user role.

#	Role	Wants to	But	Because	Feels
PS1	Event Operations Manager	Run event planning operations smoothly	Client, event, venue and vendor data cannot be handled from one place	Running events needs coordination across many areas	Frustrated, worried
PS2	Event Coordinator	Create and handle event bookings quickly	Event, client and venue details each need separate manual work	No single platform covers the full event lifecycle	Overwhelmed, frustrated
PS3	Venue Manager	Keep venue availability accurate	One venue can end up assigned to several events	Bookings need availability tracking and double-booking protection	Worried, unsure
PS4	Vendor Manager	Coordinate vendors across events	Vendor details and event assignments are managed separately	One event often needs several vendors and services	Overwhelmed, worried
PS5	Event Coordinator	Send event information and reminders on time	Reminders are sent by hand	Clients must be notified before their event	Stressed, worried
PS6	Event Administrator	Handle cancellation requests in an orderly way	Requests need approval and status changes	Cancellations must pass through a controlled approval step	Frustrated, unsure
PS7	Event Coordinator	Gather and manage client feedback	Feedback must be stored alongside event and client details	Each item relates to both an event and a client	Concerned about satisfaction
PS8	Event Operations Manager	Track operations and performance	Operational data is not visible in one place	Event data must be organised into reports and dashboards	Worried, less confident in decisions



Figure 1 – Customer problem statements

2.2 Empathy Map

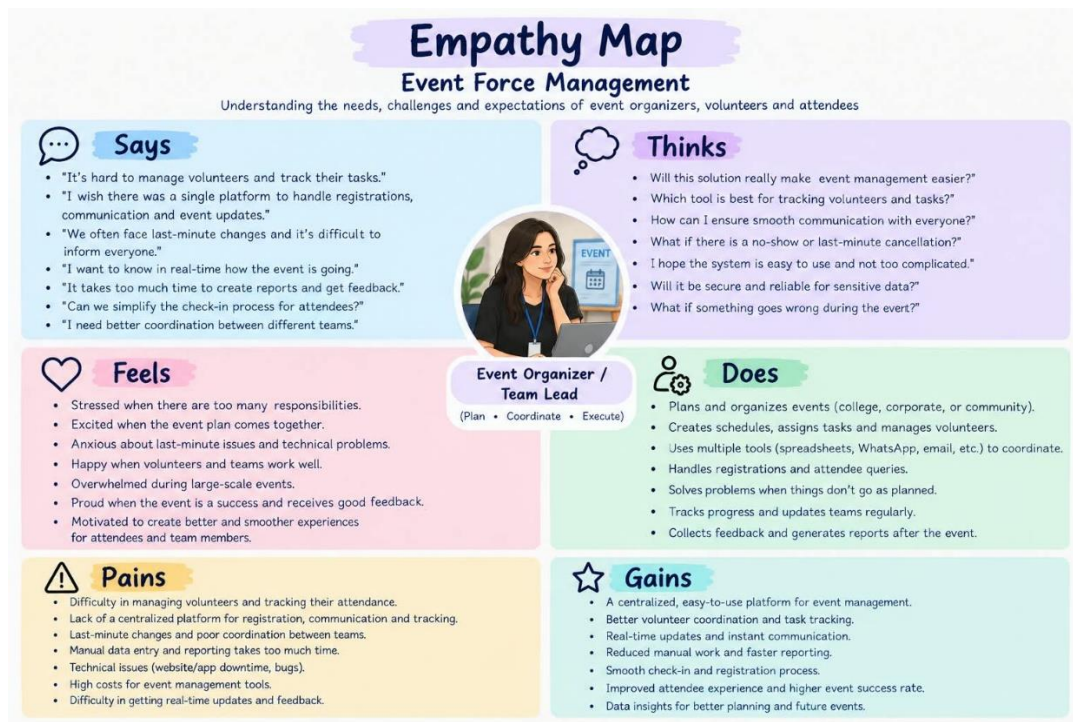


Figure 2 – Empathy map for the Event Coordinator persona

Persona: Event Coordinator

Main responsibility: handling event bookings, client requirements, venue reservations, vendor coordination, reminders and event status updates.

- **Thinks & feels:** wants events to run smoothly, wants to avoid booking clashes and to give clients a dependable experience.

- **Hears:** requirements and updates from clients, vendors, venue managers and administrators.
- **Sees:** event details, client records, venue availability, vendor assignments, cancellations and feedback.
- **Says & does:** liaises with stakeholders, updates event records, manages bookings, follows up on requirements and tracks progress.
- **Pain:** manual coordination, scattered information, the risk of double-booked venues and difficulty following many events at once.
- **Gain:** one source of event information, automated reminders and processes, reliable venue availability, easier coordination and better visibility.

2.3 Brainstorming & Idea Prioritization



Figure 3 – Brainstorming and idea prioritisation

Options Considered

Option	How it would work	Main drawback
Manual registers	Record event, client, venue and vendor details by hand	Slow and hard to keep up to date
Spreadsheets	Track bookings and events in spreadsheet files	Little automation and no real-time coordination
Standalone portal	Build a dedicated web portal for event management	Heavy development and upkeep effort
Salesforce CRM	Run event operations on the Salesforce platform	Needs proper configuration and implementation

Why Salesforce

- **Automation** – reminders, approvals, status changes and other processes run on their own.
- **Scalability** – the platform grows with more events, users, records and requirements.
- **Real-time visibility** – centralised data, reports and dashboards support monitoring.

Decision: Salesforce CRM is the best fit for a centralised, automated and scalable EventForce system.

3. Requirement Analysis

3.1 Customer Journey

Stage	Journey step	What happens
1	Enquiry & client registration	Client needs and contact details are recorded in Salesforce.
2	Event planning & booking	Event type, date, budget and status are created and maintained.
3	Venue selection & reservation	Suitable venues are shortlisted, availability is checked and reservations are handled without double booking.
4	Vendor coordination	Vendors and services are assigned to the event and their status is tracked.
5	Confirmation & communication	A confirmed event triggers automatic messages, including a reminder to the client ahead of the date.
6	Execution & monitoring	Progress is followed through Salesforce records, reports and dashboards.
7	Cancellation & approval	Cancellation requests go through the approval process, with status updates and notifications.
8	Feedback & closure	Feedback is captured after the event and completed events are updated for tracking and analysis.



Figure 4 – Customer journey map

3.2 Solution Requirements

Functional Requirements

ID	Requirement
FR-1	Event management
FR-2	Client management

ID	Requirement
FR-3	Venue management
FR-4	Vendor management
FR-5	Assigning vendors to events
FR-6	Venue availability and double-booking prevention
FR-7	Automated event reminders
FR-8	Approval of event cancellations
FR-9	Feedback management
FR-10	Reports and dashboards
FR-11	Automatic event status updates
FR-12	Data migration with the Data Import Wizard

Non-Functional Requirements

Attribute	Description
Usability	A Lightning App that makes navigation and event handling easy
Security	Access controlled through profiles, roles and permission sets
Reliability	Dependable automation, approvals and venue-booking validation
Performance	Efficient handling of event and operational data
Availability	Cloud access to Salesforce for authorised users
Scalability	Handles growth in events, clients, vendors, venues and users
Maintainability	Flows, Apex, validation rules and security can be monitored and changed
Data integrity	Validation and controlled migration keep records accurate

3.3 Data Flow

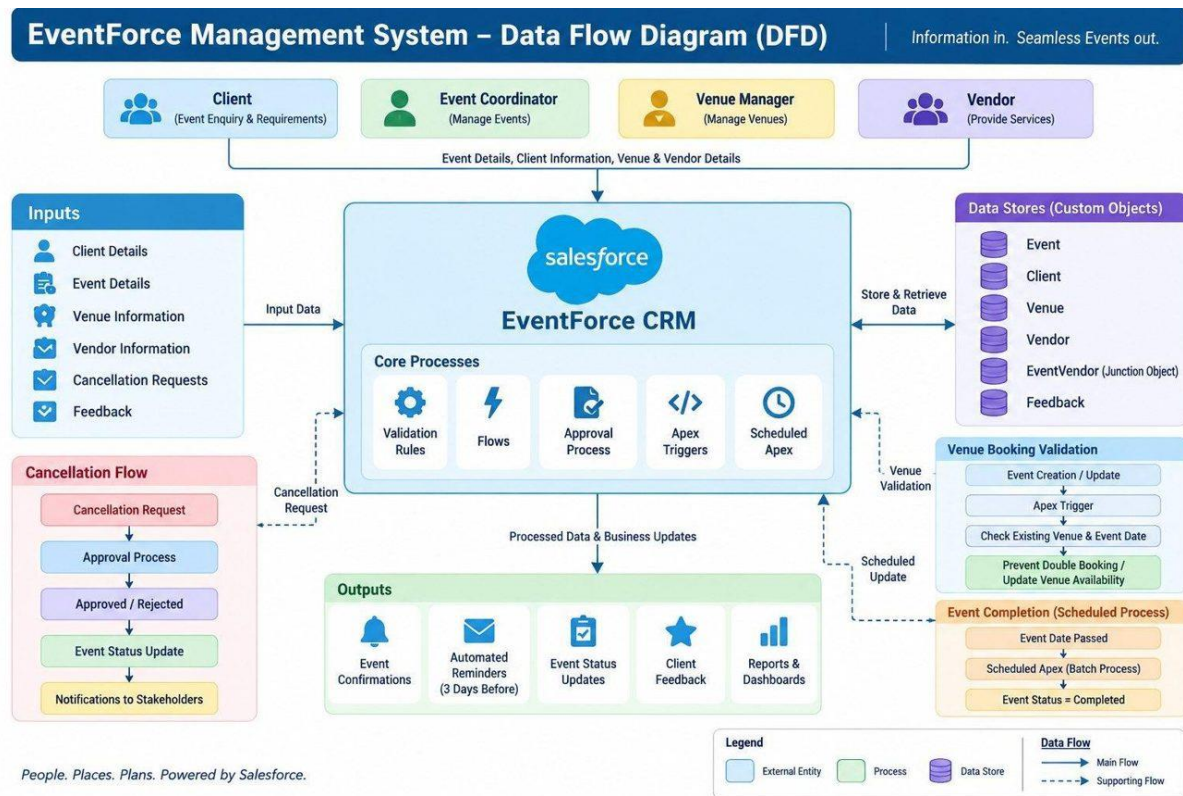


Figure 5 – Data flow diagram

Main Flow

Client or event team → enter client and event details → EventForce validates and stores the data → event, client, venue and vendor records → venue availability check → venue and vendor assignment → event confirmation → automatic reminder three days before → event execution and status updates → feedback collection → reports and dashboards.

Supporting Flows

- **Cancellation:** Event Coordinator submits a request → Approval Process → approved or rejected → event status updated → stakeholders notified.
- **Venue booking check:** Event saved → Apex trigger → VenueStatusHelper → compare against existing bookings for that venue and date → block the double booking or update venue availability.
- **Event completion:** Event date passes → Scheduled Apex starts a batch → event status becomes Completed.

Overview

Users (clients, coordinators, vendor and venue managers) → Salesforce EventForce → custom objects (Event, Client, Vendor, Venue, Feedback, EventVendor) → Flows, Approval Process and Apex → operational data and status updates → reports and dashboards → decisions.

3.4 Technology Stack

Layer	Technology
Platform	Salesforce CRM, Developer Edition
User interface	Salesforce Lightning App

Layer	Technology
Data storage	Salesforce custom objects
Business logic	Flows, formula fields, validation rules
Approvals	Salesforce Approval Process
Automation	Record-triggered Flows and Apex triggers
Background processing	Batch Apex and Scheduled Apex
Security	Profiles, roles, permission sets, sharing rules, OWD
Reporting	Reports and dashboards

4. Project Design

4.1 Problem–Solution Fit

Each operational problem identified earlier maps to a specific Salesforce capability. Custom data structures, automation, Apex logic, approvals, security and reporting work together to address the pain points of event planning.

Problem	EventForce solution	Benefit
Client, event, venue and vendor data is hard to manage centrally	Custom objects: Event, Client, Venue, Vendor, Feedback, EventVendor	Centralised, structured data
Booking involves many manual steps	Flows and automated processes	Less manual effort, faster processing
Venues may be double booked	Apex trigger and VenueStatusHelper	Availability is validated and clashes are blocked
Vendor coordination is hard to follow	EventVendor junction object and Vendor records	Clear event–vendor assignment
Clients may miss event reminders	3-Day Reminder Flow	Reminders go out automatically before confirmed events
Cancellations need controlled handling	Event_Cancellation_Process approval	Structured approvals and status updates
Feedback must tie to events	Feedback object linked to Event and Client	Organised feedback records
Operations are hard to monitor	Reports and EventForce Operations Dashboard	Better visibility for decisions
Users need different access levels	Profiles, roles, permission sets, OWD, sharing rules	Controlled, secure access



Figure 6 – Problem–solution fit

4.2 Proposed Solution

Aspect	Description
Problem	Event management that is manual and fragmented
Solution	Salesforce CRM
Innovation	Automated reminders, venue validation and cancellation approvals
Impact	Faster coordination and clearer operational visibility
Business model	Centralised event planning and management
Scalability	Supports more events, clients, vendors, venues and users

4.3 Solution Architecture

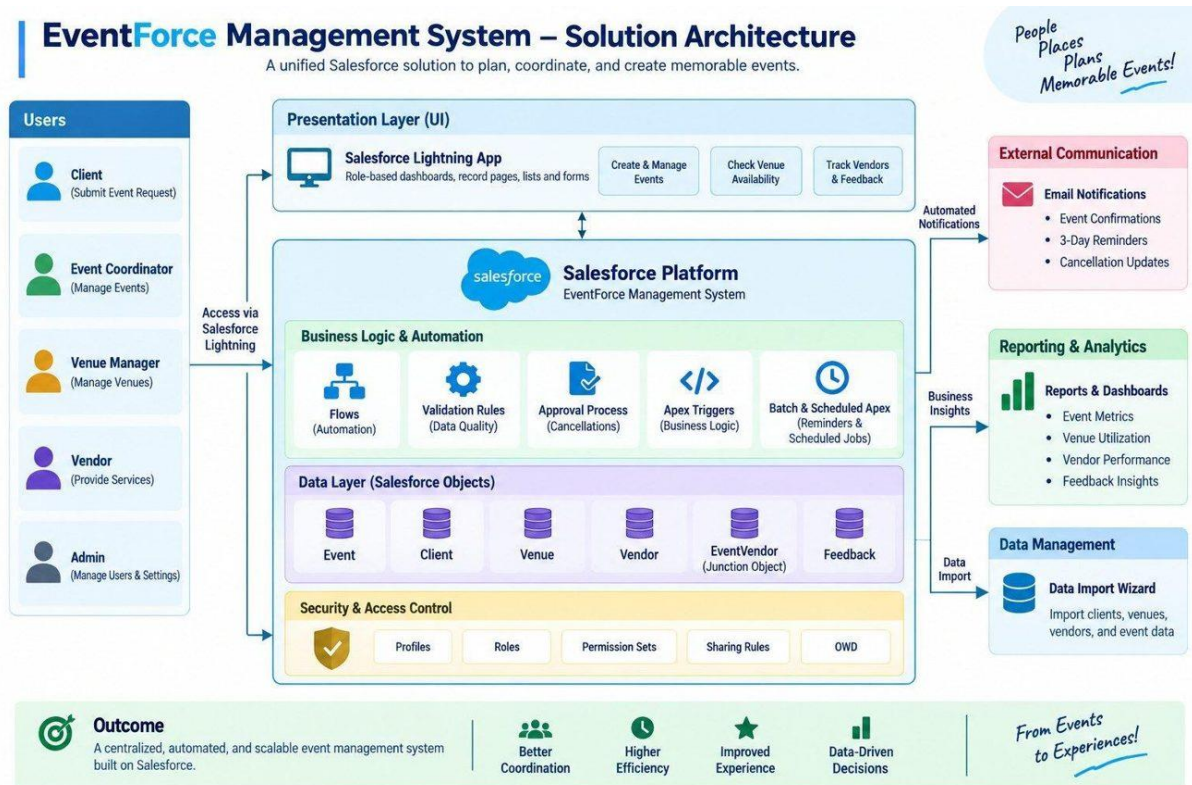


Figure 7 – Solution architecture

The solution is made up of the following components:

Data & logic	Automation & processing	Security, UI & analytics
Event, Client, Vendor, Venue objects	Flows	Profiles & roles
EventVendor junction object	Approval Process	Permission sets & sharing rules
Feedback object	Apex triggers & classes	Data Import Wizard
Validation rules	Batch Apex	Lightning App
Formula fields	Scheduled Apex	Reports & dashboards

5. Project Planning & Scheduling

5.1 Approach

- Agile methodology
- Work organised as epics → user stories → story points
- Sprint-based delivery
- Velocity used to estimate capacity

Product Backlog

Sprint	Epic	Story	Description	Pts	Priority	Owner
1	Event management	US-01	As an Event Coordinator, I want to create events with a type, date and status so event data is standardised.	3	High	Roobina J
1	Client & venue management	US-02	As an Event Coordinator, I want to link a client and venue to an event so coordination is centralised.	5	High	Pradheepa P
2	Vendor management	US-03	As a Vendor Manager, I want to assign several vendors to an event so services are coordinated.	5	High	Grace Vilma Michael
2	Notifications & automation	US-04	As a Client, I want timely reminders so I know when my event is close.	3	High	Vinoth kumar M
3	Approval management	US-05	As an Event Coordinator, I want cancellations to follow an approval process so they are controlled.	5	High	Dennish P
3	Venue management & Apex	US-06	As a Venue Manager, I want double bookings prevented so schedules stay accurate.	8	High	Roobina J
4	Security & access	US-07	As an Event Administrator, I want role-based access so users see only what they should.	5	High	Pradheepa P
4	Reports & dashboards	US-08	As Management, I want reports and dashboards so operations can be monitored.	5	Medium	Grace Vilma Michael

Sprint Tracker

Sprint	Points	Duration	Start	Planned end	Completed	Released
Sprint-1	20	6 days	24 Oct 2022	29 Oct 2022	20	29 Oct 2022
Sprint-2	20	6 days	31 Oct 2022	05 Nov 2022		
Sprint-3	20	6 days	07 Nov 2022	12 Nov 2022		
Sprint-4	20	6 days	14 Nov 2022	19 Nov 2022		

Velocity: the team planned 20 story points per sprint, 80 points across four 6-day sprints. That averages 20 points per sprint, or roughly 3.33 points per day ($20 \div 6$).

6. Project Development

6.1 Phase 1: Requirement Analysis & Planning

The aim was a single Salesforce application through which event planners manage client bookings, vendor coordination, venue reservations and feedback. Event scheduling, cancellation approvals and client reminders are automated, and reports and dashboards give live visibility.

This phase established what the system must do, who would use it and the order in which it would be built.

6.1.1 Functional Scope

- Custom objects: Event, Client, Vendor, Venue, Feedback and Event Vendor (junction).
- Relationships: Lookup, Master-Detail and many-to-many (Event ↔ Vendor).
- Flows for reminders, feedback notifications and approval updates.
- Validation rules and formula fields for clean data and automatic event budgets.
- An approval process for cancellations, with email alerts to coordinators and clients.
- Profiles, roles and permission sets for user-level security.
- Reports and dashboards for event summaries, vendor performance and upcoming events.

6.1.2 Stakeholder Mapping

Stakeholder	Role	Profile	Main needs
Event Administrator	Event Admin	Event Admin	Full oversight, security and monitoring
Event Coordinator	Event Coordinator	Event Coordinator	Scheduling, client communication, approvals
Vendor Manager	Vendor Manager	Vendor Manager	Vendor records and assigned services
Client	Client	Client Profile	View own events, submit feedback

6.1.3 Execution Roadmap

Step	Activity	Outcome
1	Object and field setup	Event, Client, Vendor, Venue, Feedback and Event Vendor objects created
2	Automation Flows	Record-triggered flow for event reminders
3	Validation rules and formula fields	Accurate data and automatic budget calculation
4	Profiles, roles, permission sets	User access and record visibility configured
5	Approval process	Cancellation workflow with notifications
6	Apex classes and triggers	Venue availability updates and double-booking prevention
7	Reports and dashboards	Analytics on upcoming events and performance

6.2 Phase 2: Backend Development & Configuration

This phase built the working core of the system. Declarative tools (record-triggered Flows, validation rules, formula fields and approval processes) handle booking confirmations, reminders and cancellations,

while Apex classes and triggers cover venue availability updates and double-booking prevention. Work was split into the milestones below.

Milestone 1 – Salesforce Developer Account

Salesforce is a customer-success platform for selling, servicing, marketing, analysing and connecting with customers. A free Developer Edition org was set up to build and test the whole project.

Activity 1 – Create the account

1. Roobina J P
2. Enter first name, last name and email, choose Developer as the role, enter the college name as company, select India as country and add the postal code.
3. Provide a unique username in email format (for example username@organization.com); it does not need to be a working mailbox.
4. Sign me up.

Activity 2 – Activate the account

1. Open the verification email (it can take 5–10 minutes) and click Verify Account.
2. Set a password, answer the security question and click Change Password.
3. You land on the Salesforce Setup home page.

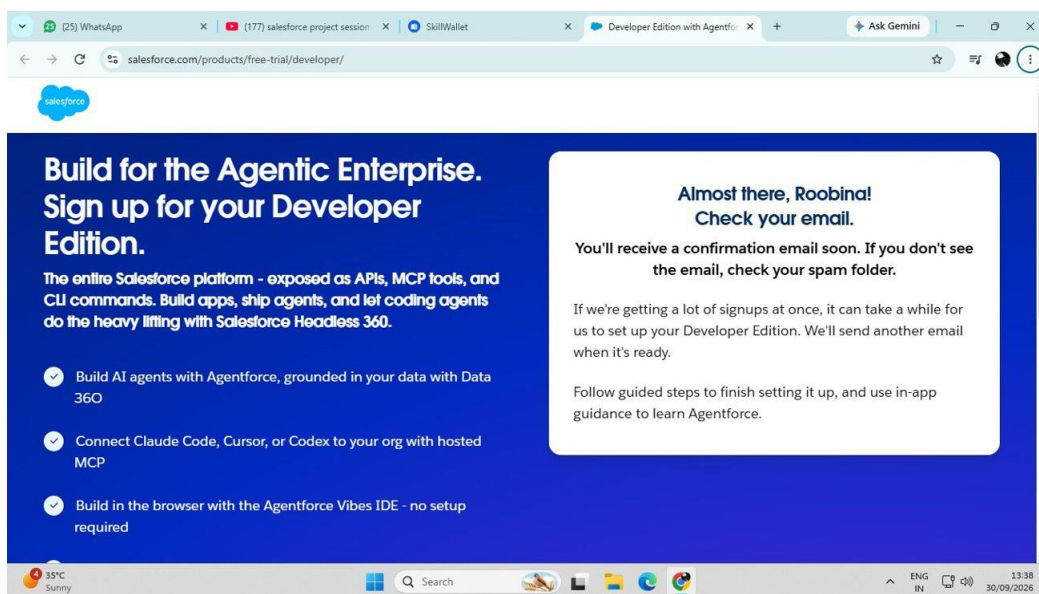


Figure 8 – Developer Edition sign-up confirmation

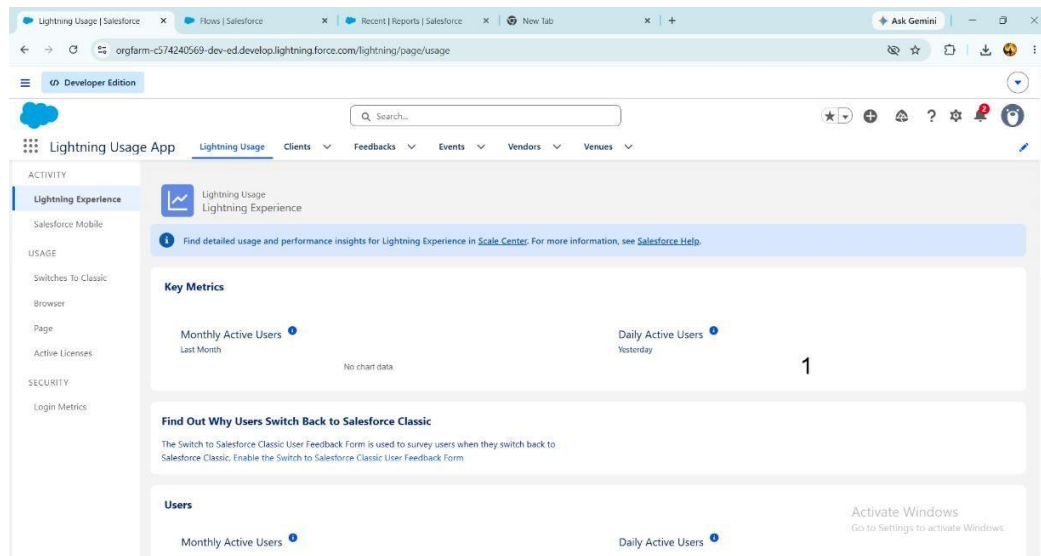


Figure 9 – Lightning Usage app after first login

6.2.1 Data Architecture

The data model has five main custom objects and one junction object. An object works like a database table: fields are its columns and records are its rows.

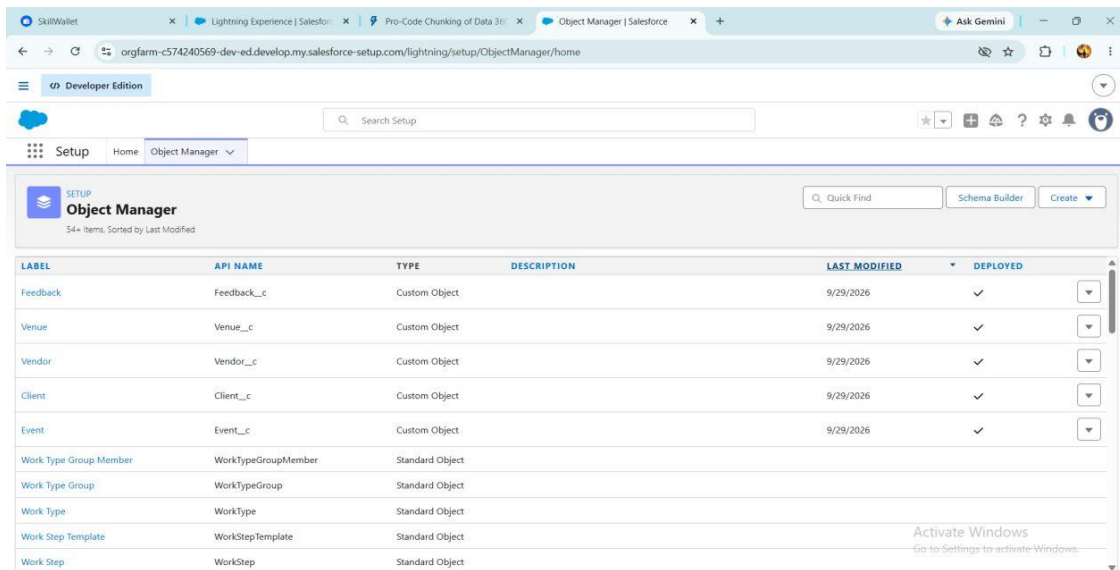
Object	API name	Record name	Purpose
Event	Event__c	Event Name (Text)	Event details, status and budget
Client	Client__c	Client Name (Text)	Client contact information
Vendor	Vendor__c	Vendor Name (Text)	Vendor details and services
Venue	Venue__c	Venue Name (Text)	Venue details and availability
Feedback	Feedback__c	Feedback ID (Auto Number, F-{0000})	Client ratings and comments
Event Vendor	EventVendor__c	Auto-generated	Junction linking Event and Vendor

Milestone 2 – Creating Custom Objects

Each object was made through Setup → Object Manager → Create → Custom Object. The Event object used these options:

- Label Event, plural label Events, record name Event Name, data type Text.
- Optional features: Allow Reports, Allow Activities, Track Field History.
- Deployment status Deployed; search status Allow Search.
- Notes and Attachments related list added to the default page layout.

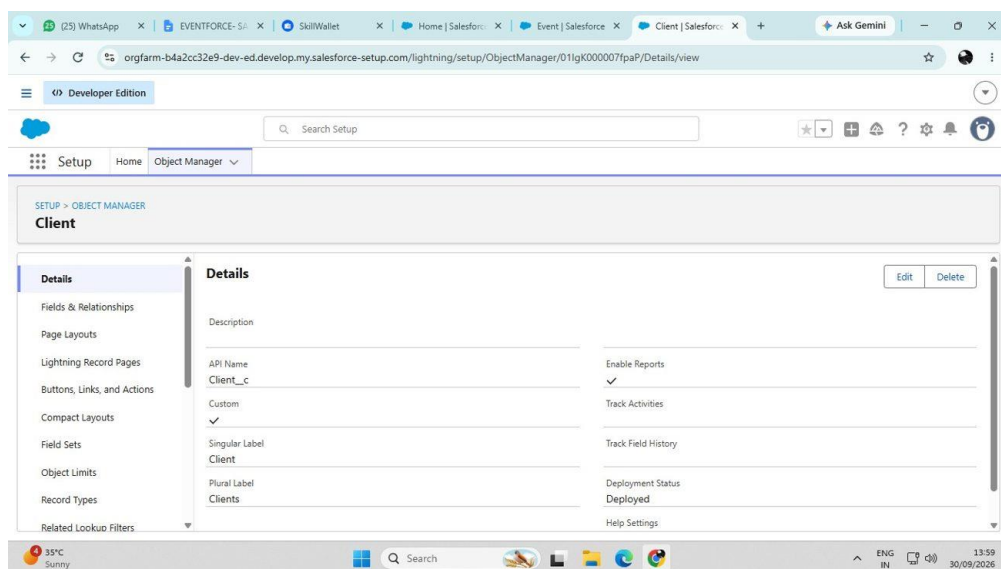
Client, Vendor and Venue were created the same way (text record name, with Allow Reports, Allow Search and the Notes and Attachments list). Feedback uses an Auto Number record name formatted F-{0000}, starting at 0. The Event Vendor junction object was created separately, as described under Relationships.



The screenshot shows the Salesforce Object Manager interface. At the top, there's a search bar and a 'Quick Find' button. Below the header, a table lists various objects. The table has columns for Label, API Name, Type, Description, Last Modified, and Deployed. Custom objects like Feedback__c, Venue__c, Vendor__c, Client__c, and Event__c are listed, all with a deployment status of 'Deployed'. Standard objects like Work Type Group Member, Work Type Group, Work Type, Work Step Template, and Work Step are also listed.

LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED
Feedback	Feedback__c	Custom Object		9/29/2026	✓
Venue	Venue__c	Custom Object		9/29/2026	✓
Vendor	Vendor__c	Custom Object		9/29/2026	✓
Client	Client__c	Custom Object		9/29/2026	✓
Event	Event__c	Custom Object		9/29/2026	✓
Work Type Group Member	WorkTypeGroupMember	Standard Object			
Work Type Group	WorkTypeGroup	Standard Object			
Work Type	WorkType	Standard Object			
Work Step Template	WorkStepTemplate	Standard Object			
Work Step	WorkStep	Standard Object			

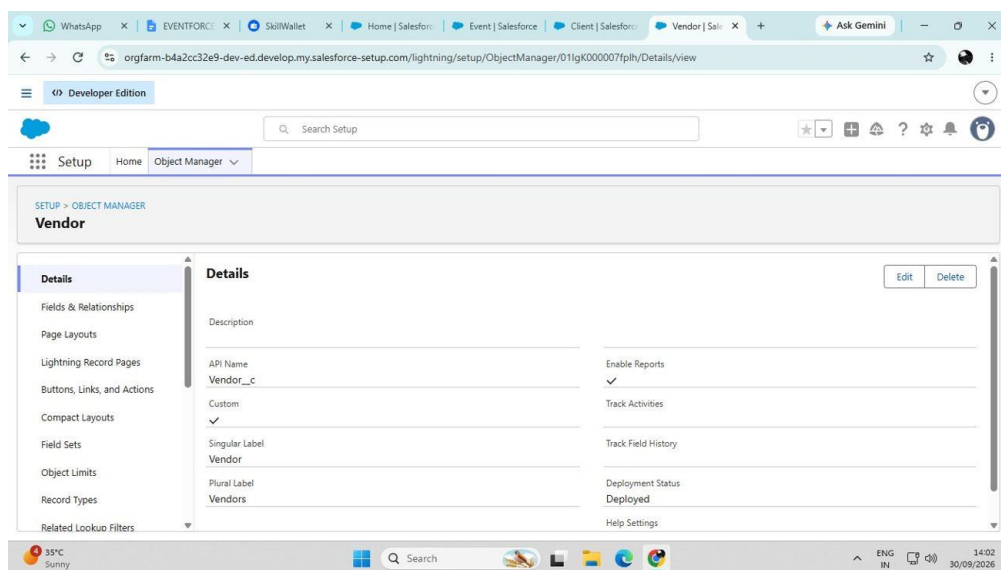
Figure 10 – Custom objects listed in Object Manager



The screenshot shows the details page for the 'Client' object in Salesforce Object Manager. The left sidebar contains a navigation menu with options like 'Details', 'Fields & Relationships', 'Page Layouts', 'Lightning Record Pages', 'Buttons, Links, and Actions', 'Compact Layouts', 'Field Sets', 'Object Limits', 'Record Types', and 'Related Lookup Filters'. The main content area displays the 'Details' for the 'Client' object, including fields for API Name (Client__c), Singular Label (Client), Plural Label (Clients), and Deployment Status (Deployed). There are also checkboxes for 'Enable Reports' and 'Track Activities'.

Field	Value
Description	
API Name	Client__c
Custom	✓
Singular Label	Client
Plural Label	Clients
Deployment Status	Deployed
Enable Reports	✓
Track Activities	
Track Field History	
Help Settings	

Figure 11 – Client object details (Client__c)



The screenshot shows the details page for the 'Vendor' object in Salesforce Object Manager. The left sidebar contains a navigation menu with options like 'Details', 'Fields & Relationships', 'Page Layouts', 'Lightning Record Pages', 'Buttons, Links, and Actions', 'Compact Layouts', 'Field Sets', 'Object Limits', 'Record Types', and 'Related Lookup Filters'. The main content area displays the 'Details' for the 'Vendor' object, including fields for API Name (Vendor__c), Singular Label (Vendor), Plural Label (Vendors), and Deployment Status (Deployed). There are also checkboxes for 'Enable Reports' and 'Track Activities'.

Field	Value
Description	
API Name	Vendor__c
Custom	✓
Singular Label	Vendor
Plural Label	Vendors
Deployment Status	Deployed
Enable Reports	✓
Track Activities	
Track Field History	
Help Settings	

Figure 12 – Vendor object details (Vendor__c)

Milestone 3 – Custom Tabs

Tabs present each object's data at a glance. For Event, Client, Vendor, Venue and Feedback, a tab was added via Setup → Tabs → Custom Object Tabs → New: choose the object, pick a tab style icon, accept the defaults (Next, Next) and Save.

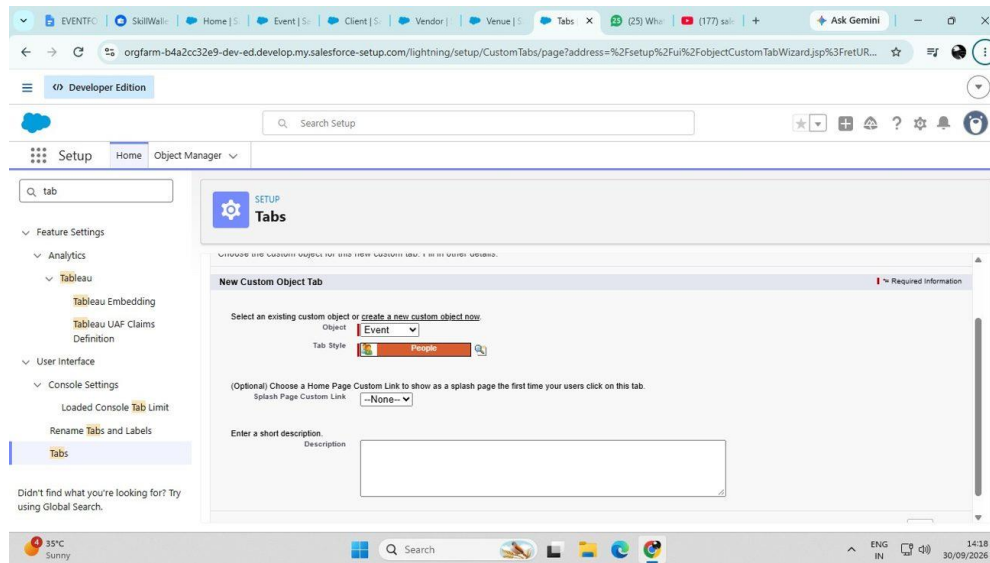


Figure 13 – Creating the Event custom tab

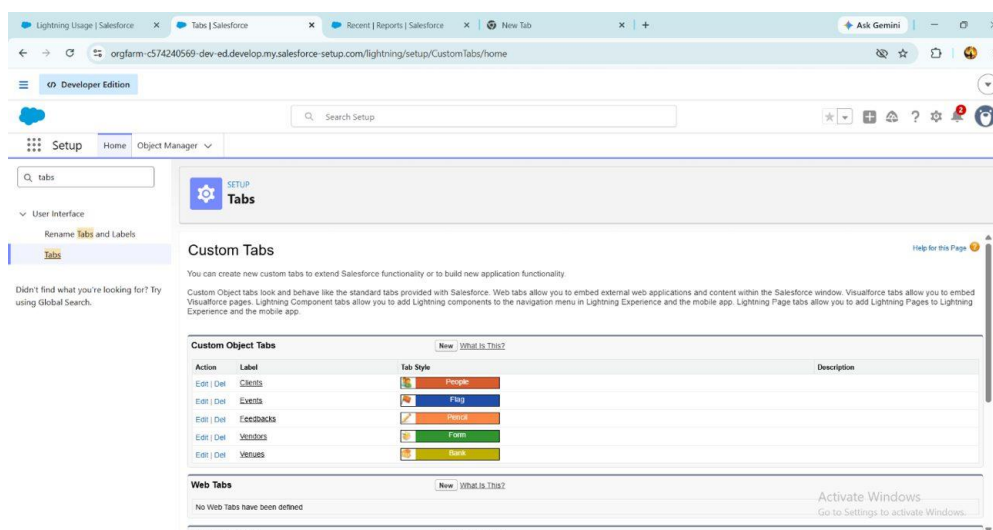


Figure 14 – Custom object tabs created

Milestone 4 – Fields

Fields were added to every object using Object Manager → Fields & Relationships → New with a suitable data type.

Object	Field	Type	Values / notes
Event	Event Type	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event	Event Date	Date	Date the event takes place
Event	Event Status	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event	Event Budget	Formula (Currency)	Derived from Event Type (see section 6.2.3)

Object	Field	Type	Values / notes
Client	Email	Email	Client email address
Client	Phone	Phone	Client contact number
Client	Address	Text Area	Client address
Client	Country	Picklist	India, Philippines, Saudi Arabia, USA, Switzerland, Thailand, Maldives, Dubai, Sri Lanka, Bangkok, Singapore, Myanmar, France, Nepal
Client	City	Picklist	Hyderabad, Mumbai, Delhi, Bangalore, Chennai, Alaminos, Angeles City, Palayan, Quezon City, San Juan, Riyadh, Jeddah, Tayma, Dammam, Medina, New York, Los Angeles, Chicago, Houston, Columbus, Geneva, Basel, Lausanne, Bern, Lucerne
Vendor	Service Type	Multi-select picklist	Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Vendor	Email	Email	Vendor email address
Vendor	Phone	Phone	Vendor contact number
Vendor	Status	Picklist	Available, Booked, Cancelled
Venue	Address	Text Area	Venue address
Venue	Location	URL	Map link to the venue
Venue	Capacity	Number	Maximum number of guests
Venue	Availability Status	Picklist	Available, Reserved, Not Available
Feedback	Rating	Picklist	1, 2, 3, 4, 5
Feedback	Comments	Long Text Area	Client comments

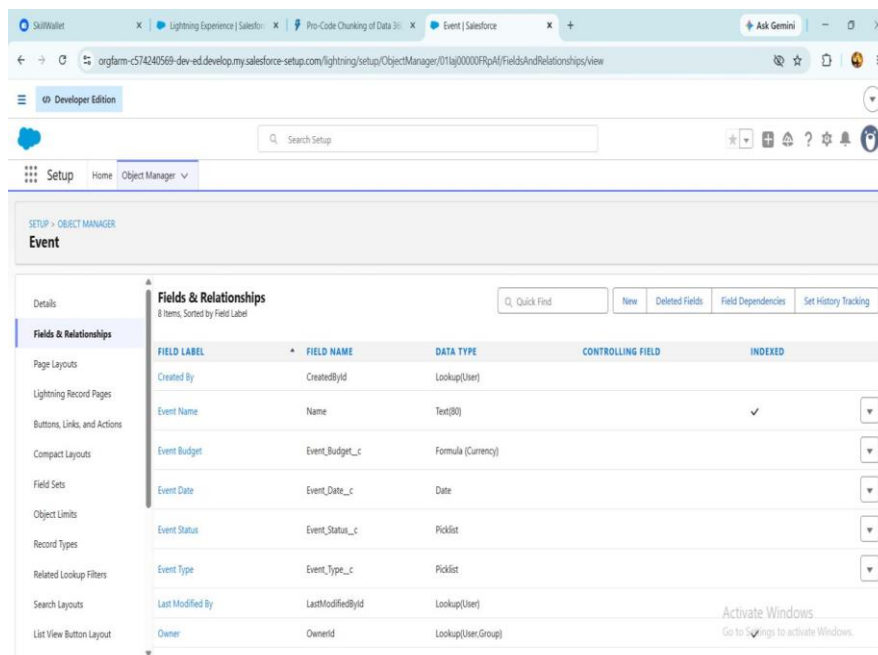


Figure 15 – Fields and relationships on the Event object

Relationships

Relationships tie the objects together. A Lookup links records loosely, whereas a Master-Detail creates a tight parent-child dependency. Salesforce has no direct many-to-many relationship, so the Event Vendor junction object, carrying two Master-Detail fields, links Event and Vendor.

Relationship	Type	Field label
Event → Client	Lookup	Client
Event → Venue	Lookup	Venue
Feedback → Event	Lookup	Event
Feedback → Client	Lookup	Client
EventVendor → Event	Master-Detail	Event
EventVendor → Vendor	Master-Detail	Vendor

Creating the lookups

1. In Setup open Object Manager and choose the Event object.
2. Select Fields & Relationships → New → Lookup Relationship → Next.
3. Pick the related object (Venue), click Next, enter the field label, then continue through Next, Next, Next and Save.
4. Repeat for the Client lookup on Event and for the Event and Client lookups on Feedback.

Creating the many-to-many link (Event ↔ Vendor)

1. Go to Object Manager → Create → Custom Object, with label Event Vendor and object name EventVendor__c.
2. On that object choose Fields & Relationships → New → Master-Detail Relationship, select Event, label it Event and click Save & New.
3. Add a second Master-Detail relationship to Vendor, label it Vendor and Save.

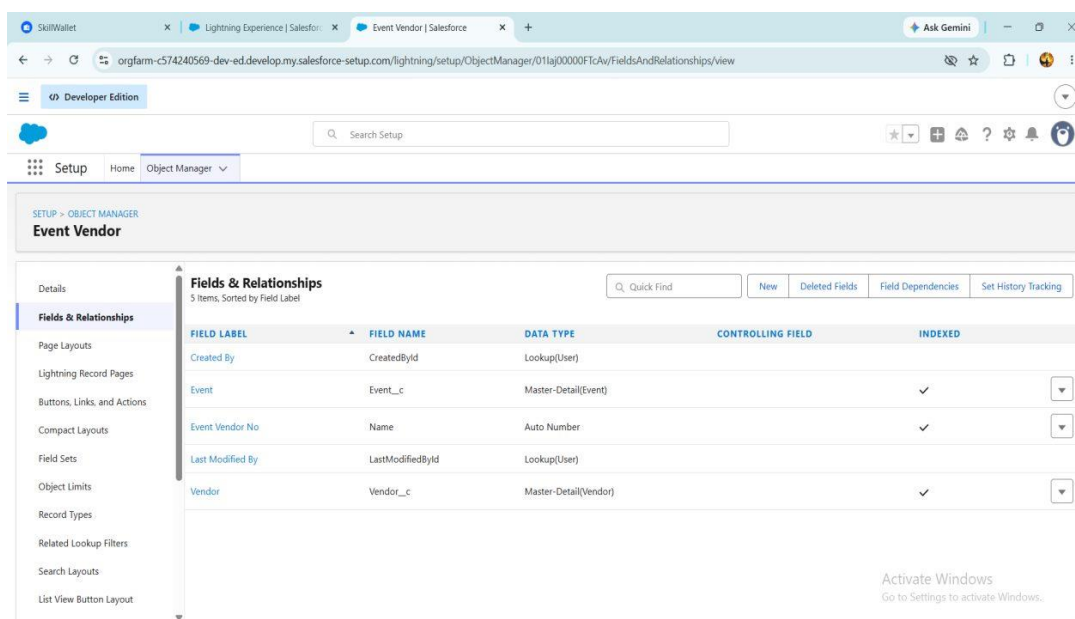


Figure 16 – Event Vendor junction object fields

Lookup filter (Feedback → Event)

Scenario: after a client is chosen on a Feedback record, the Event lookup should list only that client's events. If Ramesh is selected, only Ramesh's events appear.

1. Open Object Manager → Feedback → Fields & Relationships → Event (Event__c) and click Edit.
2. In the Lookup Filter section click Show Filter Settings.
3. Set the criteria: Event: Client (Client__c) equals Field → Feedback: Client (Client__c).
4. Tick Enable this filter and click Save.

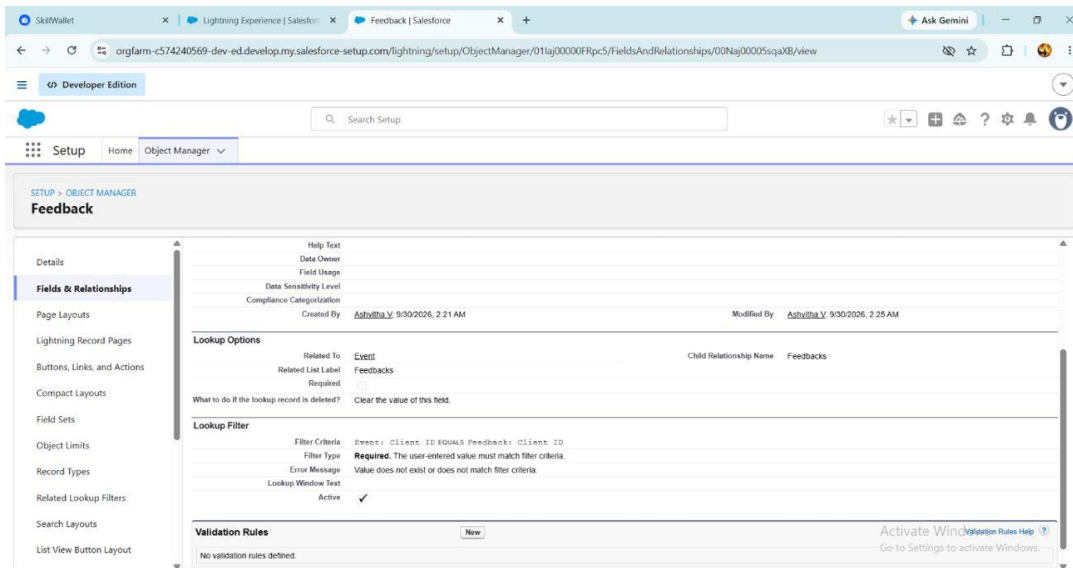


Figure 17 – Lookup filter on the Feedback → Event field

6.2.2 Formula Field – Event Budget

Event Budget is a Currency formula field on Event. It works out the budget from the Event Type automatically, which removes manual-entry errors. Go to Fields & Relationships → New → Formula, label it “Event Budget”, choose Currency, paste the formula below, click Check Syntax, then Next, Next, Save.

```
CASE(
  TEXT(Event_Type__c),
  "Wedding", 50000,
  "Corporate", 30000,
  "Birthday", 10000,
  "Anniversary", 20000,
  "Festival", 60000,
  "Concert", 40000,
  "Other", 15000,
  0
)
```

Event type	Budget
Wedding	50,000
Corporate	30,000
Birthday	10,000
Anniversary	20,000
Festival	60,000
Concert	40,000
Other	15,000

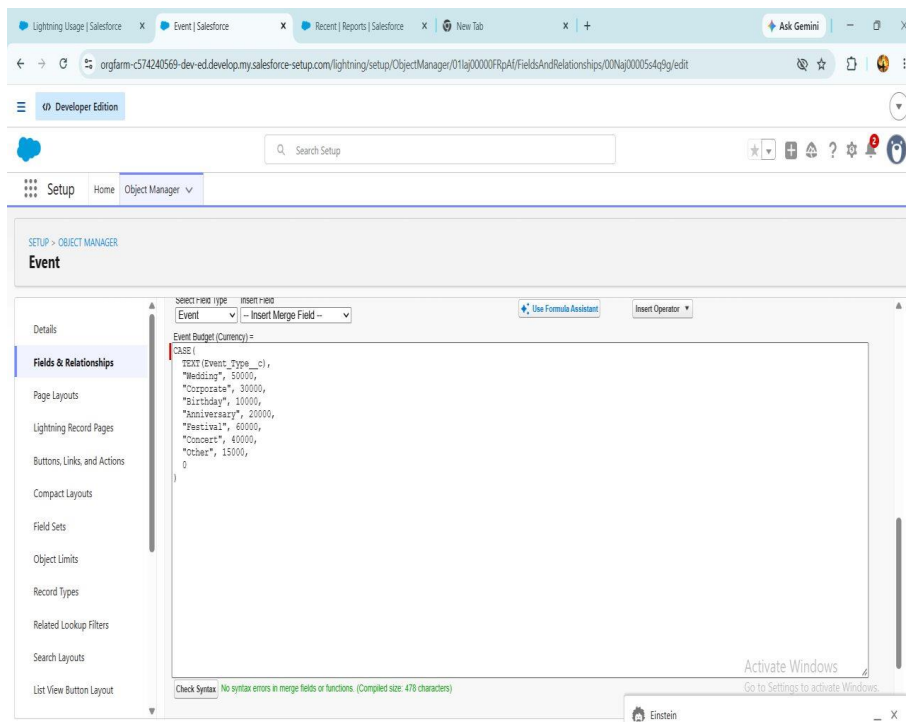


Figure 18 – Event Budget formula and syntax check

6.2.3 Milestone 5 – Validation Rule

A validation rule runs when a record is saved and blocks the save if the data fails the criteria. One was created on Client so that only properly formatted email addresses are stored.

1. Setup → Object Manager → Client → Validation Rules → New.
2. Rule name Email_Valid_Address, with Active ticked.
3. Enter the error condition formula shown below.
4. Set the error message to “Please Enter Valid Email Address”, error location Field → Email, and Save.

```
NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9._]+\.[a-zA-Z]{2,}$"))
```

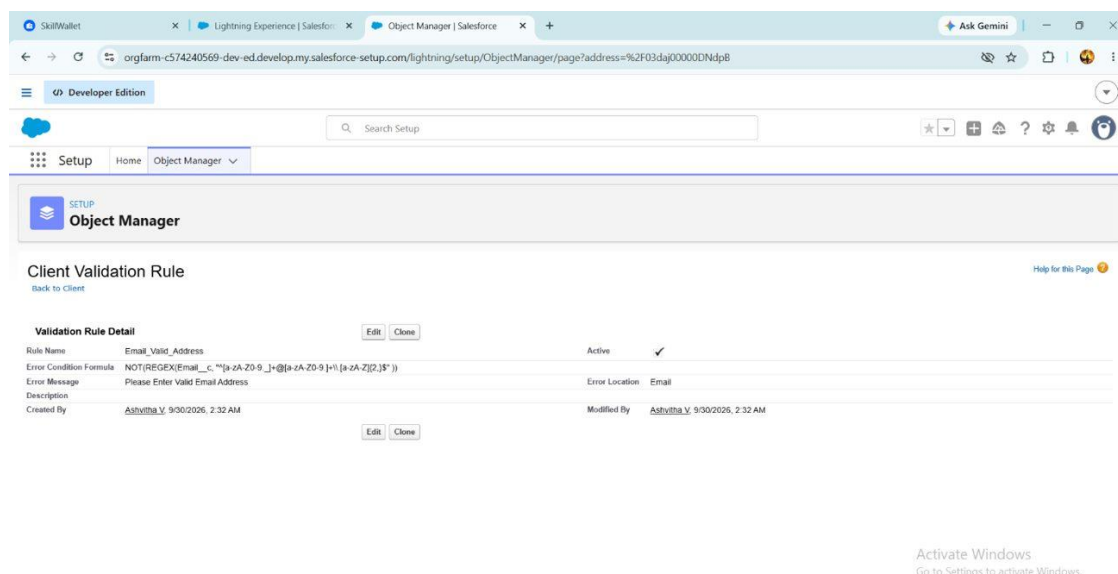


Figure 19 – Client validation rule detail

6.2.4 Milestone 6 – Approval Process (Event Cancellation)

This process makes sure an event is canceled only after a manager approves it, and that all parties are notified automatically. The Profiles, Roles and Users milestones (6.4.1–6.4.3) must be completed before the approval process is built.

Step A – Classic email templates. Four Text-type classic templates were created in Setup → Email Templates → Classic Email Templates → New Template (folder Unfiled Public Email Templates, UTF-8).

Template	Subject	Used for
Event Cancellation Request Notification	Approval Request: Cancel Event {!Event__c.Name}	Alert when an approval request is submitted
Email to Client After Cancellation	Event Cancellation Notice – {!Event__c.Name}	Tells the client the cancellation is approved
Approval Notification to Event Owner	Event Cancellation Approved: {!Event__c.Name}	Tells the owner of approval
Rejection Notification to Event Owner	Event Cancellation Rejected: {!Event__c.Name}	Tells the owner of rejection

Sample body – Event Cancellation Request Notification

Hello!

An approval request has been submitted to cancel the event:

- Event Name: {!Event__c.Name}
- Event Date: {!Event__c.Event_Date__c}

Thank you,
EventForce Management System

Sample body – Approval Notification to Event Owner

Hello {!Event__c.OwnerFullName},

Your cancellation request for the event {!Event__c.Name} has been approved.

Event Date: {!Event__c.Event_Date__c}

The event status has been updated to Canceled.

Thank you,
EventForce Management System

The rejection template follows the same layout, saying the request was rejected and that the event status reverted to Confirmed. The client template states that the event (name and date) has been canceled and invites the client to contact the team with questions.

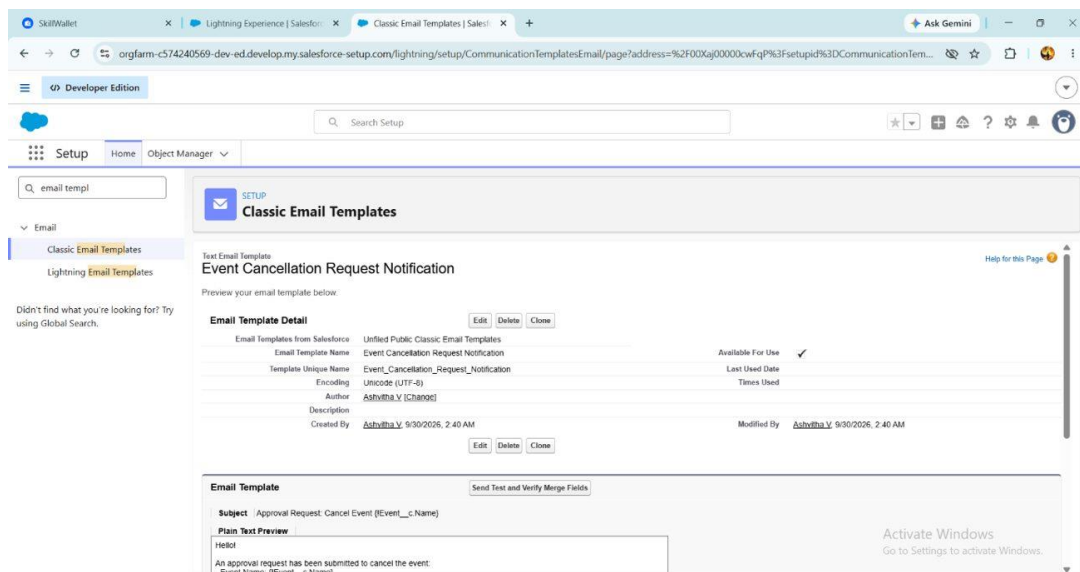


Figure 20 – Classic email template: Event Cancellation Request Notification

Step B – Create the approval process

1. Setup → Approval Processes → choose the Event object → Create New Approval Process → Use Standard Setup Wizard.
2. Basic settings: name Event_Cancellation_Process (the unique name fills in automatically).
3. Entry criteria: Event: Event Status equals Pending Cancellation.
4. Approver: assign automatically to a user or queue, such as the submitter's manager (make sure a manager is set on the User record).
5. Email template: Event Cancellation Request Notification.
6. Approval page fields: Event Name, Owner, Client, Last Modified By, Created By, Event Budget, Event Date, Event Status, Event Type, Venue.
7. Initial submitters: Event Owner. Click Save.

Step C – Actions

Stage	Action	Configuration
Initial submission	Email alert	Event Cancellation Request alert; template Event Cancellation Request Notification; recipient role Event Coordinator; sender is the current user
Final approval	Field update	Event Status Update: set Event Status to Canceled

Stage	Action	Configuration
Final approval	Email alert	Cancellation Approved alert; template Approval Notification to Event Owner; recipient Event Owner
Final rejection	Email alert	Cancellation Rejected alert; template Rejection Notification to Event Owner; recipient Event Owner
Final rejection	Field update	Event Status To Rejected: set Event Status to Rejected

Step D – Approval step and activation

1. Click Add New Step, name it Manager Review and let all records enter the step; click Next.
2. Assign the approver automatically (a specific or related user) and click Save.
3. Open View Approval Process Detail Page and click Activate.

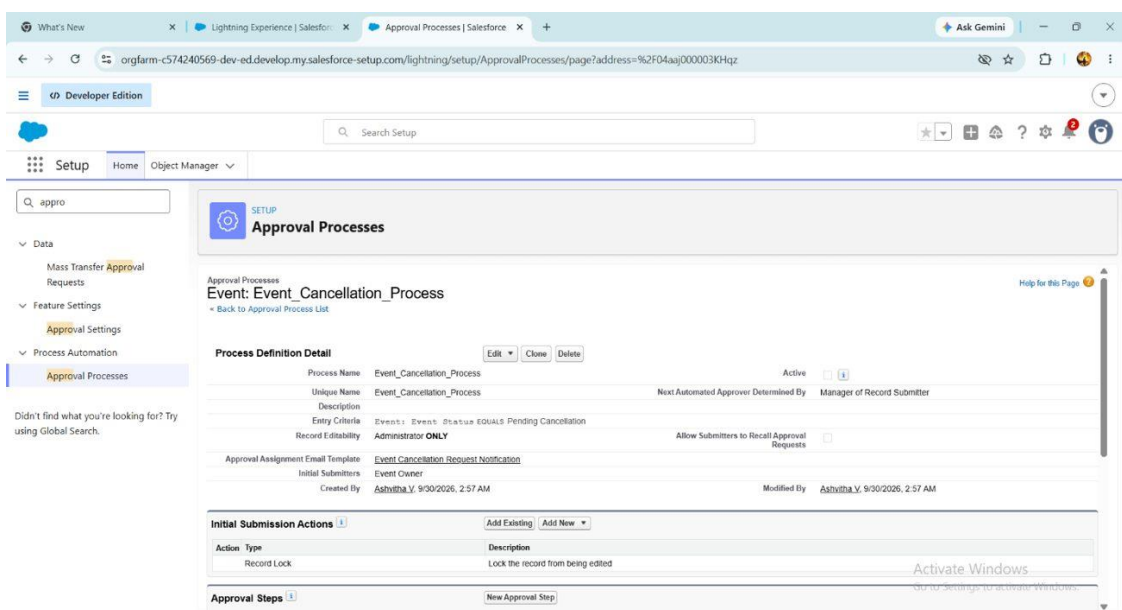


Figure 21 – Event_Cancellation_Process approval process

6.2.5 Milestone 7 – Flow: 3-Day Reminder Email

Scenario: when an Event is created or updated with Event Status = Confirmed and the client has a valid email address, the system emails the client a reminder three days before the Event Date.

1. Setup → Flows → New Flow → Record-Triggered Flow → Create.
2. Object Event; trigger when a record is created or updated.
3. Condition: Event Status equals Confirmed. Optimise for Actions and Related Records, then click Done.
4. Click Start → Add Schedule Path. Path name 3_Days_Before; time source Event__c: Event Date; offset 3 days before.
5. From the toolbox choose New Resource → Text Template. API name Subject, plain text: "Reminder: Your Event {!\$Record.Name} is in 3 days".
6. Create a second text template with API name Body (content below).
7. Under the 3_Days_Before path click + → Action → Send Email, labelled Alert_Client_3Day_Reminder.
8. Recipient address list: Client Email; CC address list: Event Owner.
9. Compose the email content with the Subject resource as subject and the Body resource as body.

10. Save with the flow label “Client Reminder - 3 Days Before”, then Save & Activate.

Body text template

```
Hello {!$Record.Client__r.Name},

This is a reminder that your event "{!$Record.Name}" will take place on
{!$Record.Event_Date__c}.

Venue: {!$Record.Venue__r.Name}
Type: {!$Record.Event_Type__c}

We look forward to seeing you!
- EventForce Team
```

6.2.6 Milestone 8 – Apex Class & Trigger: Venue Status

Scenario: when an Event with a Venue is created or updated, the venue’s Availability Status becomes Reserved if the event is Confirmed, and returns to Available if the event is Canceled. The logic sits in a helper class called from a thin trigger, which is the recommended practice.

Apex class VenueStatusHelper – gear icon → Developer Console → File → New → Apex Class, then save with Ctrl+S.

```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c,
Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}
```

Apex trigger EventTrigger13 – File → New → Apex Trigger, sObject Event__c.

```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Triiger.new);
}
```

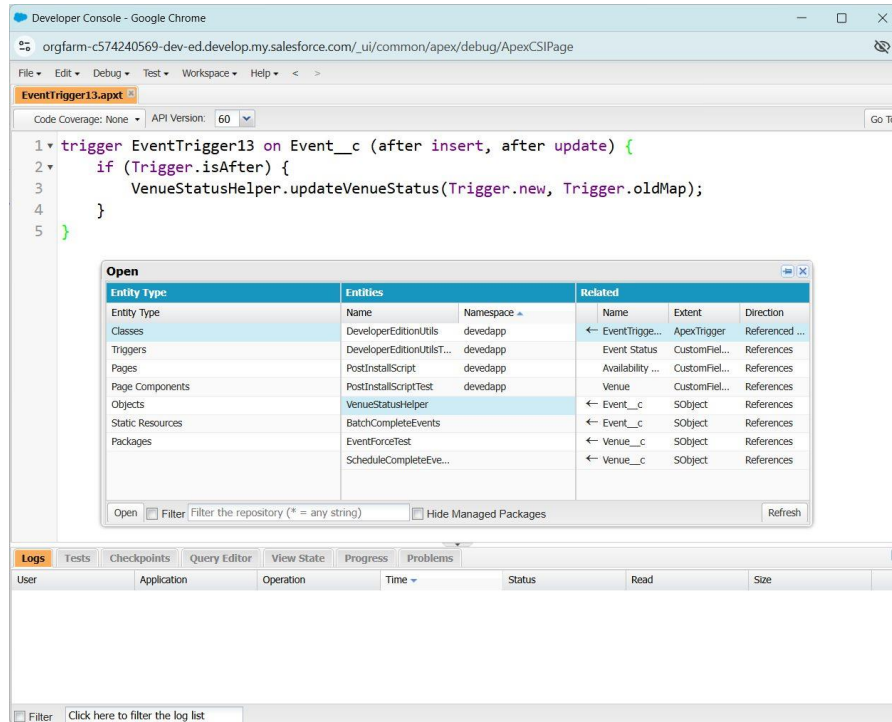


Figure 22 – EventTrigger13 in the Developer Console

6.2.7 Milestone 9 – Apex Trigger: Prevent Double Booking

Scenario: two events must not share a venue on the same date. This before-insert / before-update trigger gathers the venues involved, fetches the existing events at those venues in a single (bulk-safe) query, and raises an error when another event already holds that date.

```

trigger PreventDoubleBooking on Event__c (before insert, before update) {
    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }

    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
                      FROM Event__c WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }

    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
                    ev.addError('This Venue is already booked on this date.');

```

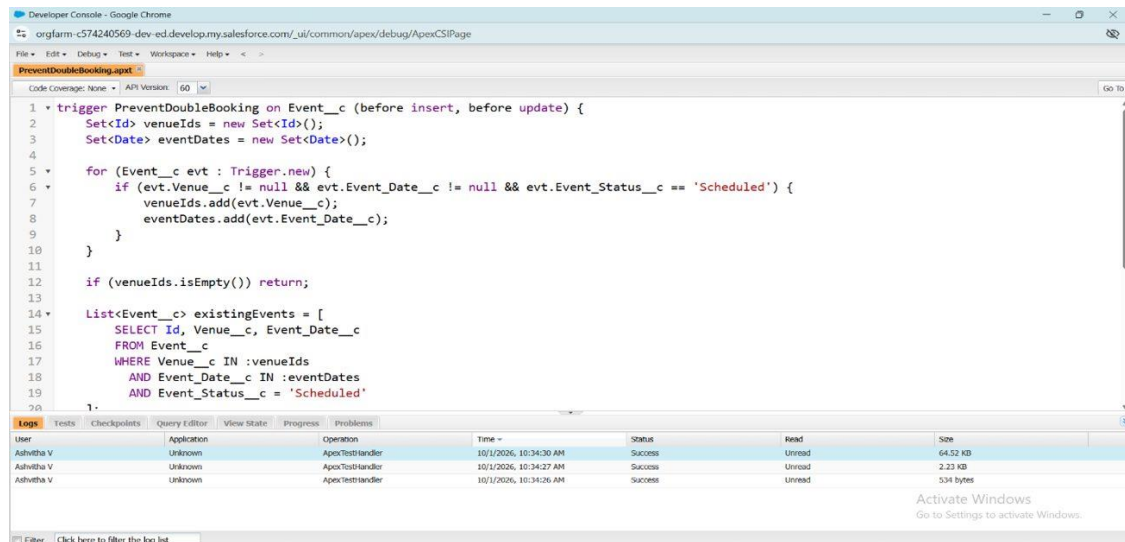


Figure 23 – PreventDoubleBooking trigger in the Developer Console

6.2.8 Milestone 10 – Batch & Schedulable Apex

Scenario: every night, mark all past events as Completed. A Batch Apex class handles the records in chunks and a Schedulable class launches it automatically.

Batch class BatchCompleteEvents

```
global class BatchCompleteEvents implements Database.Batchable<Object> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator(
            'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c ' +
            'WHERE Event_Date__c < TODAY AND Event_Status__c != \'Completed\''
        );
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){
            ev.Event_Status__c = 'Completed';
        }
        update scope;
    }
    global void finish(Database.BatchableContext bc){
        System.debug('Past events marked as Completed');
    }
}
```

Schedulable class ScheduleCompleteEvents

```
global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}
```

Scheduling the job

1. Setup → Apex Classes → click Schedule Apex.
2. Job name Daily Event Completion; Apex class ScheduleCompleteEvents.
3. Frequency Weekly with every day selected; preferred start time 8:00 PM.
4. Click Save. The batch now runs each night at 8 PM and marks past events Completed.

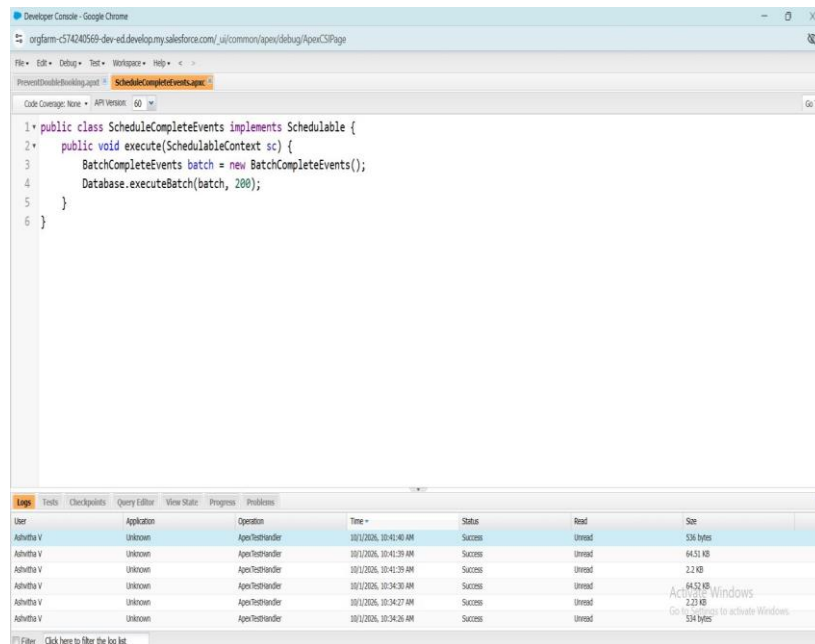


Figure 24 – ScheduleCompleteEvents class

6.3 Phase 3: UI/UX Development & Customization

This phase delivered an easy-to-use interface: a custom Lightning App that gathers every module into one workspace, plus reports and dashboards showing event schedules, vendor performance and client feedback.

Milestone 11 – The Lightning App

An app is a set of tabs working together under one name and logo. The Event Planner app was built as follows:

1. Setup → App Manager → New Lightning App. Name it Event Planner, upload an image and click Next.
2. Keep the default app options and utility items; click Next.
3. Move Event, Client, Vendor, Venue, Feedback, Reports and Dashboards from Available Items to Selected Items; click Next.
4. Move System Administrator (and any other needed profiles) to Selected Profiles.
5. Click Save & Finish, then confirm by searching Event Planner in the App Launcher.

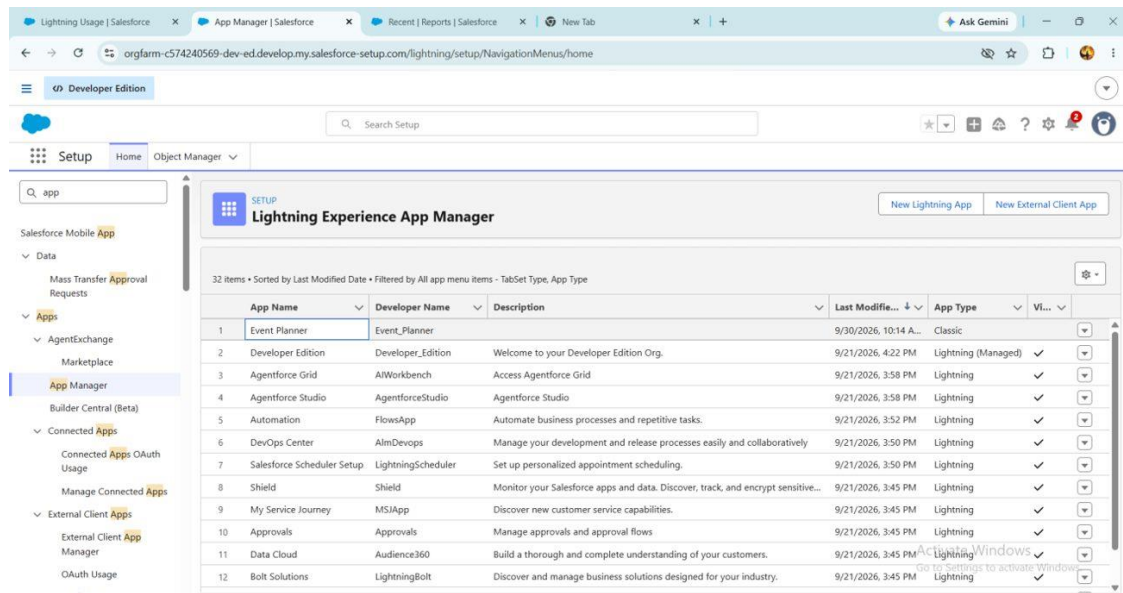


Figure 25 – Event Planner in the Lightning Experience App Manager

Milestone 12 – Reports

Report folder. In the App Launcher open Reports → New Folder, label it EventForce Report and Save. From All Folders open the folder's drop-down → Share, share with the Event Coordinator and Vendor Manager roles at View access, then Share and Done.

Report: Upcoming Events by Month. It shows upcoming events with their counts and budgets per month. At least 10 records were created in every object before building it.

1. Reports → New Report; category All; choose Events (or Events with Client & Venue); Start Report.
2. Columns: Event Name, Event Date, Event Type, Client Name, Venue Name, Event Budget.
3. Filters: Show Me = All Events; Event Date ≥ TODAY (or Next 30 Days); Event Status ≠ Canceled.
4. Group rows by Event Date, by Calendar Month.
5. Summaries: Row Count and SUM of Event Budget.
6. Chart: column or line, X-axis Event Date (Calendar Month), Y-axis Row Count or SUM(Event Budget).
7. Save as Upcoming Events by Month in the EventForce Report folder and click Run.

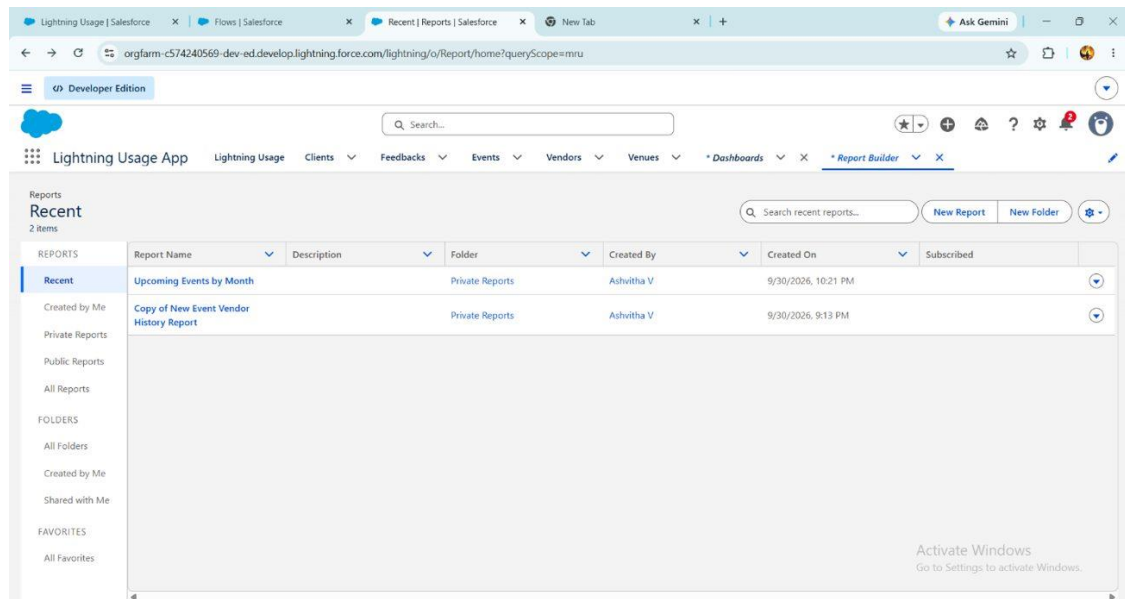


Figure 26 – Upcoming Events by Month in the report list

Milestone 13 – Dashboards

1. In the Event Planner app open Dashboards → New Dashboard.
2. Name it EventForce Operations Dashboard and click Create.
3. Click + Widget and pick the Upcoming Events by Month report.
4. Chart type Donut, max values displayed 6, subtitle Upcoming Events Overview, widget theme Light.
5. Click Save.

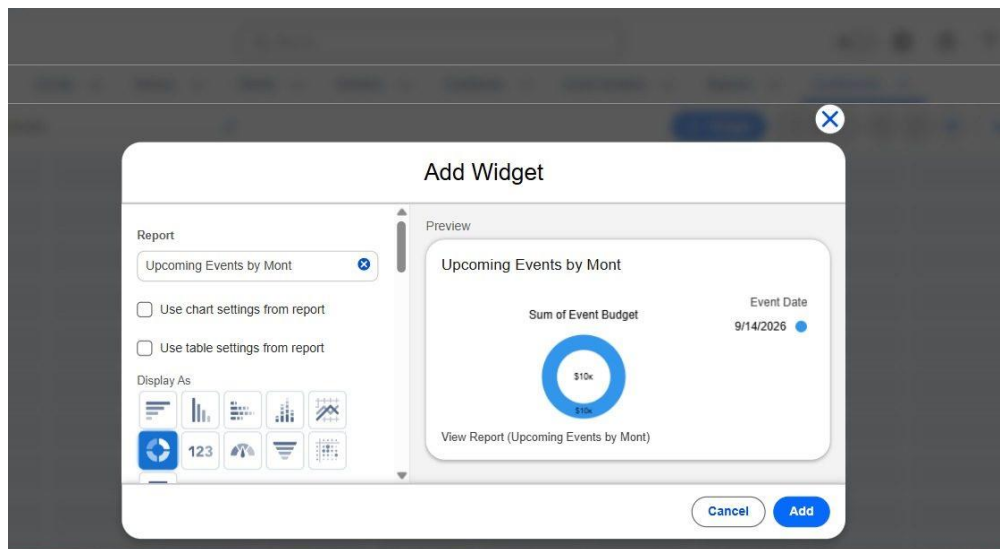


Figure 27 – Adding the Upcoming Events by Month donut widget

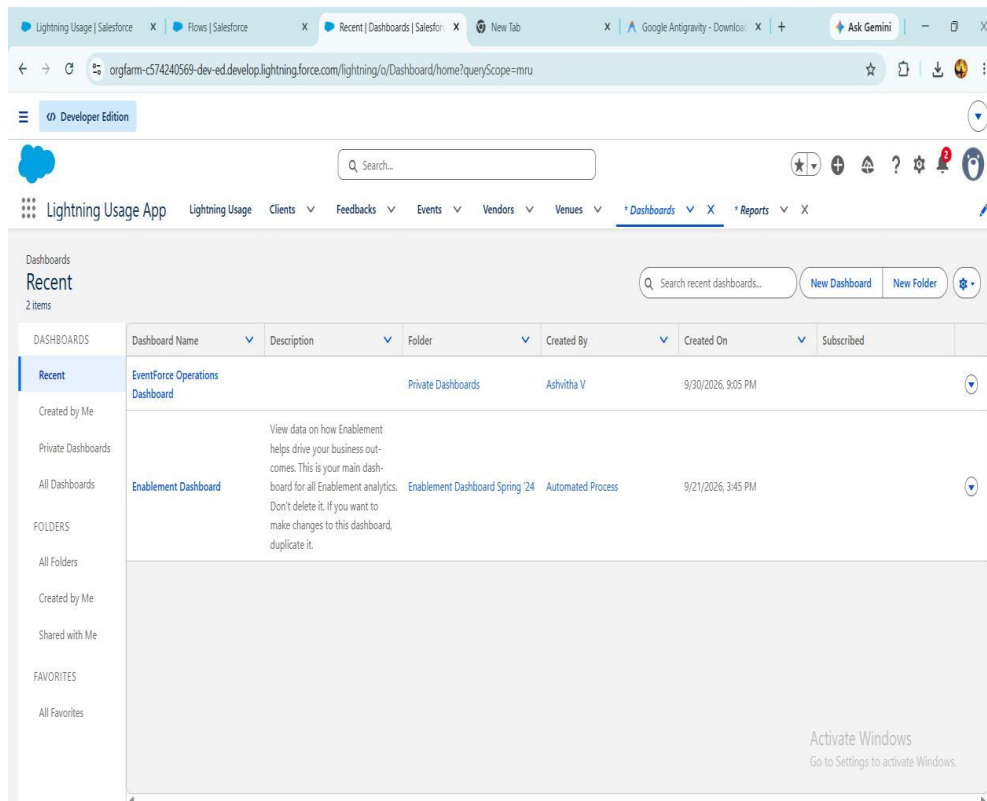


Figure 28 – Dashboards list including EventForce Operations Dashboard

6.4 Phase 4: Data Migration, Testing & Security

This phase made the system ready for use: loading validated data, creating users and applying role-based access. Records for Events, Clients, Vendors, Venues and Feedback were imported with the Data Import Wizard after checks to avoid duplicates. Four account types (Event Admin, Event Coordinator, Vendor Manager and Client) were created with matching profiles and roles. Organization-Wide Defaults were set to Private and then opened up through sharing rules.

Milestone 14 – Profiles

A profile bundles the settings and permissions that decide what a user can do: object and field access, tab and app settings, login hours and so on. Each user has exactly one profile, but one profile can serve many users.

Event Admin. Created by cloning System Administrator (Setup → Profiles → New Profile → existing profile System Administrator → name Event Admin). The clone carries Modify All Data, Customize Application and full CRUD on all objects, which suits an administrator.

Other profiles. Event Coordinator, Vendor Manager and Client Profile were cloned from Standard Platform User and edited: the default custom app is Event Planner, object permissions are as below, the session times out after 2 hours of inactivity, passwords never expire and the minimum length is 8.

Object	Event Coordinator	Vendor Manager	Client Profile
Event	Create, Read, Edit, Delete	Read only	Read only (own events)
Client	Read, Edit	Read only	—
Vendor	Read	Create, Read, Edit, Delete	No access
Venue	Read	Read only	No access
Feedback	Create, Read	Read only	Create, Read

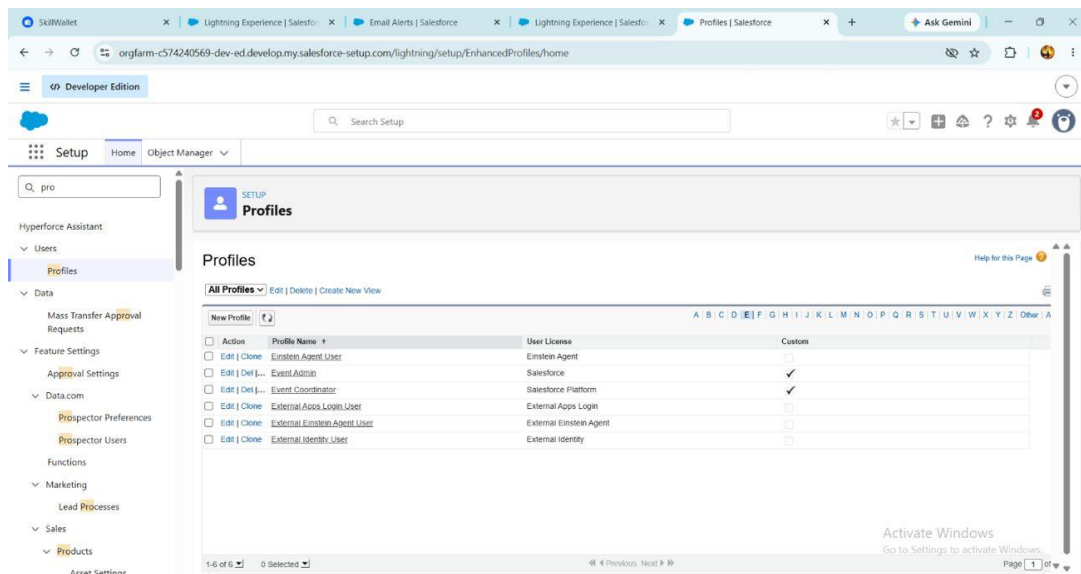


Figure 29 – Profiles page in Setup

Milestone 15 – Roles

Roles define the reporting hierarchy and let record visibility flow upward. The Event Admin role was placed under CEO (Setup → Roles → Set Up Roles → Expand All → Add Role) and three roles were added beneath it.

Role	Reports to
Event Admin	CEO
Event Coordinator	Event Admin
Vendor Manager	Event Admin
Client	Event Admin

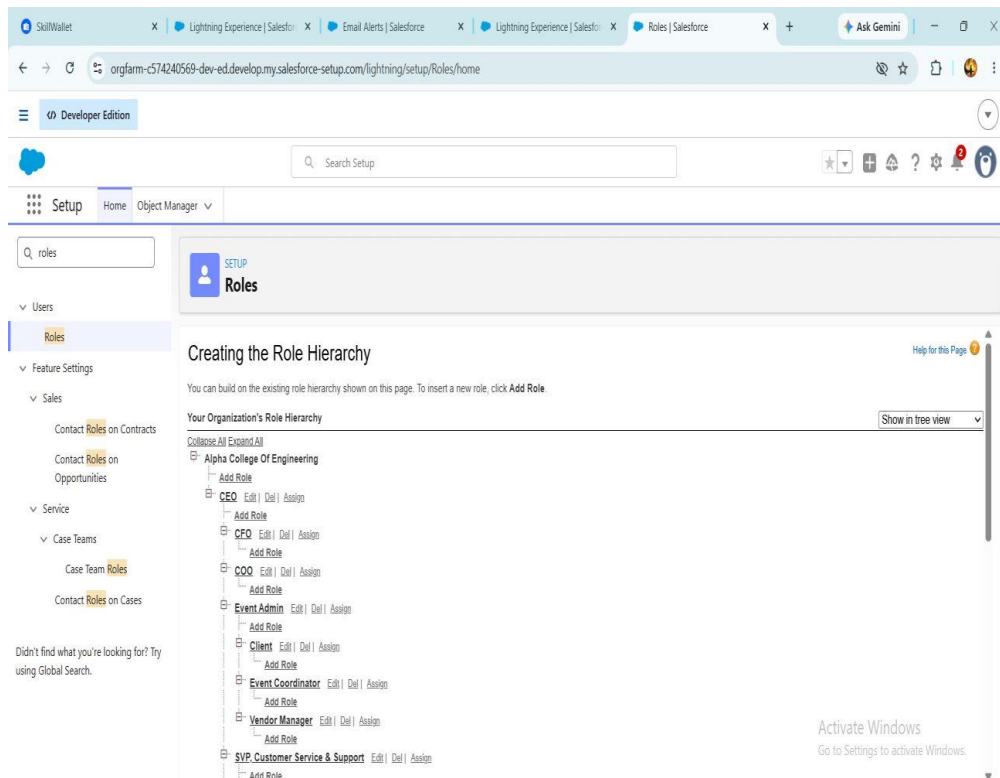


Figure 30 – Role hierarchy

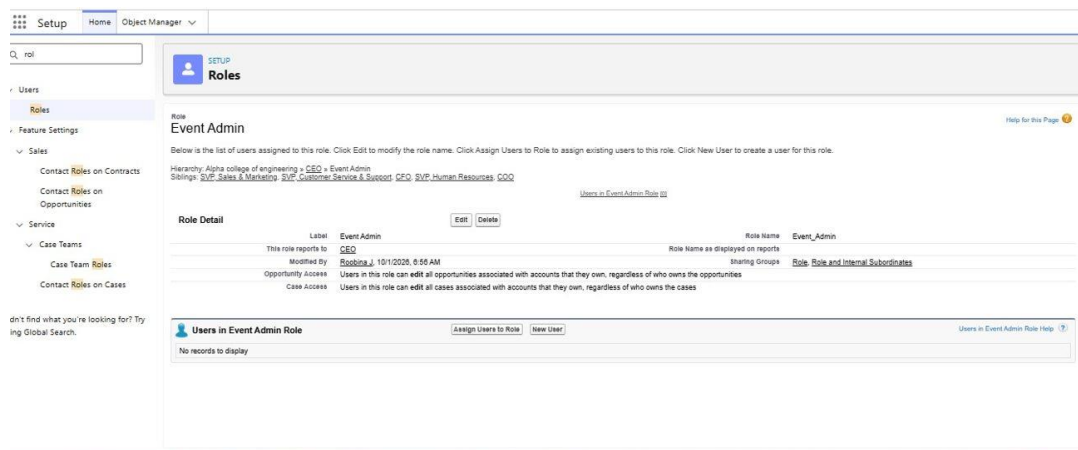


Figure 31 – Event Admin role detail

Milestone 16 – Users

A user is anyone who logs in to Salesforce. Users were created from Setup → Users → New User. The first user received the Event Admin role, Salesforce license and Event Admin profile; the others used the Salesforce Platform license with the matching role and profile.

User type	Role	License	Profile
Event Admin	Event Admin	Salesforce	Event Admin
Event Coordinator	Event Coordinator	Salesforce Platform	Event Coordinator
Vendor Manager	Vendor Manager	Salesforce Platform	Vendor Manager
Client	Client	Salesforce Platform	Client Profile

A Developer Edition org has only a few Salesforce licenses. If none are free, deactivate an existing user holding one or change the license type.

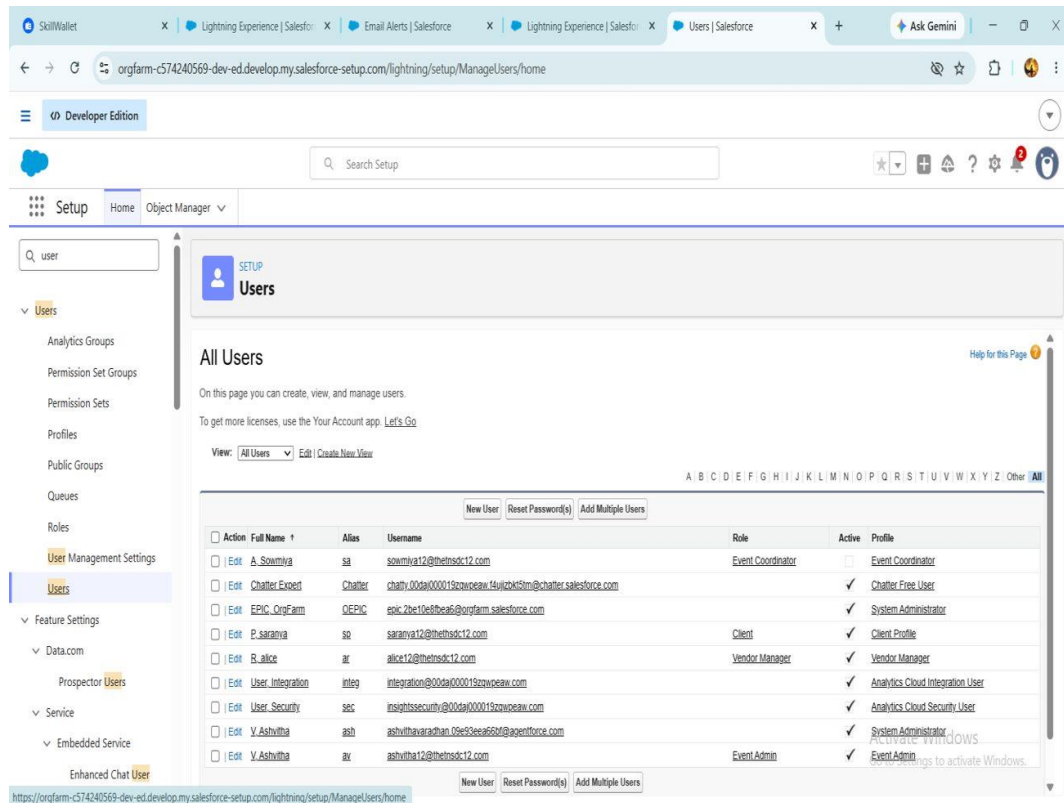


Figure 32 – All Users list

Milestone 17 – Permission Set

A permission set adds permissions without touching a user's profile. A Feedback Manager permission set was created to give selected users full access to Feedback.

1. Setup → Permission Sets → New; label it Feedback Manager and Save.
2. Open Object Settings → Feedback → Edit, enable Read, Create, Edit and Delete, and Save.
3. Click Manage Assignments → Add Assignment, select the Event Coordinator user, then Next → Assign.

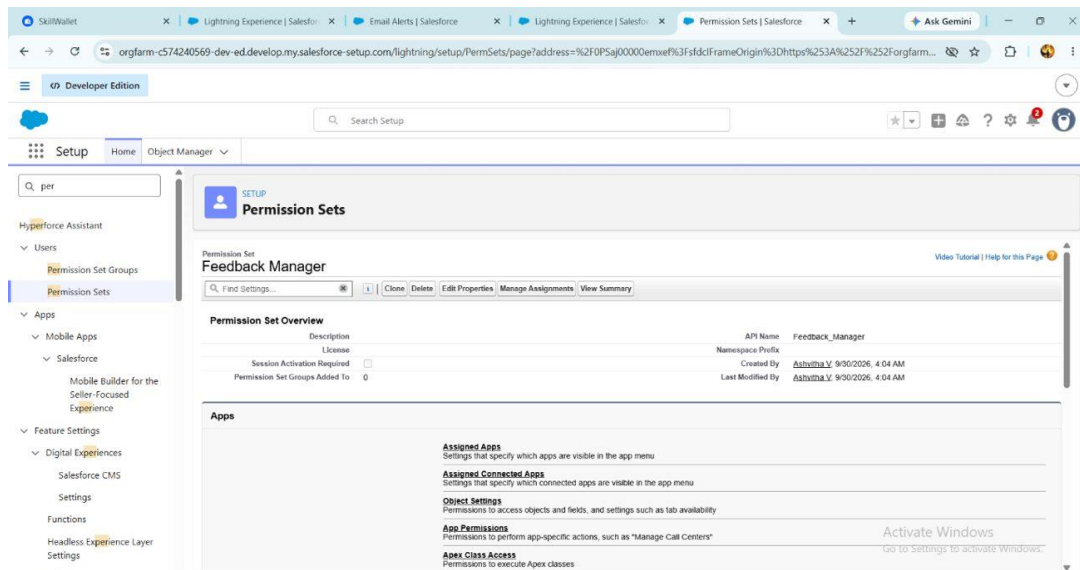


Figure 33 – Feedback Manager permission set

Milestone 18 – Sharing Settings

Organization-Wide Defaults (OWD) set the baseline record access, which can then be widened through the role hierarchy, sharing rules, manual sharing or Apex sharing. Sharing rules can only extend access, never restrict it.

Configuring OWDennish P

1. Setup → Sharing Settings → Edit (Organization-Wide Defaults).
2. For Event set Default Internal Access and Default External Access to Private.
3. Click Save.

Sharing rule: Event_Sharing_For_Vendors

1. In Sharing Settings find the Event sharing rules and click New.
2. Label it Event_Sharing_For_Vendors (the rule name fills in automatically).
3. Records to share: owned by members of Roles → Event Coordinator.
4. Share with: Roles → Vendor Manager.
5. Access level Read Only; click Save.

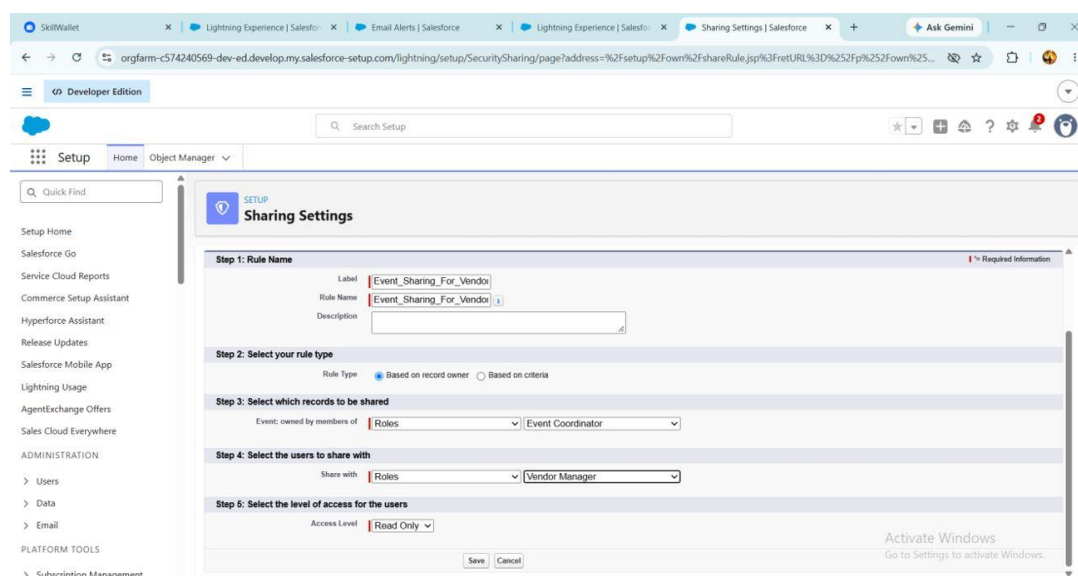


Figure 34 – Event_Sharing_For_Vendors sharing rule

6.5 Phase 5: Deployment, Documentation & Maintenance

6.5.1 Deployment Strategy

Everything was configured and tested inside a Salesforce Developer Edition org. The purpose of this phase is to show how a real Salesforce solution is readied for release after configuration, testing and security setup. No production deployment was carried out, because Developer Edition orgs are standalone and not linked to production instances; real deployments use sandboxes, change sets or DevOps tooling.

In an enterprise, the recommended path would be:

1. Build and unit-test in a Developer or Developer Pro sandbox.
2. Promote the metadata (objects, fields, flows, Apex, approval process, reports, dashboards) to a Partial or Full sandbox for UAT using change sets or Salesforce DX / Git-based CI/CD.
3. Run Apex tests (at least 75% code coverage is required to deploy to production) and validate the deployment.
4. Deploy during a low-usage window, then activate flows, approval processes and scheduled jobs and assign permission sets.
5. Load production data, train users and provide post-go-live support.

Note: Apex test classes were outside the scope of this academic build. Before any production deployment, tests for VenueStatusHelper, PreventDoubleBooking, BatchCompleteEvents and ScheduleCompleteEvents must be written to reach 75% coverage.

6.5.2 Maintenance, Monitoring & Troubleshooting

Regular upkeep keeps event, client, vendor and feedback data accurate and the automations healthy.

Activity	Approach
Monitoring	Review Flow error emails, Apex Jobs, Scheduled Jobs and pending approvals
Access issues	Check profiles, permission sets, roles and sharing rules to fix permission conflicts
Debugging	Use debug logs and the Developer Console for Apex triggers and Flow logic
Rule tuning	Adjust validation rules and sharing settings as needs change

Activity	Approach
Data quality	Run duplicate checks and periodic cleansing; look for anomalies in reports
Releases	Read the seasonal release notes and retest critical automations after each release

Common Issues

Issue	Likely cause	Resolution
User cannot see the Event Planner app or a tab	Profile or permission set lacks app/tab access	Enable app/tab visibility in the profile or permission set
Approval cannot be submitted	No manager/approver set, or entry criteria not met	Set a manager on the User record and confirm status is Pending Cancellation
Reminder email not sent	Flow inactive, status not Confirmed, or client email blank	Activate the flow; check status and client email
Event cannot be saved	Double-booking trigger fired	Choose another date or venue
Cannot create a user	No free Salesforce license in the org	Deactivate an unused user or change the license type

6.5.3 Documentation Summary

This report records the purpose, data model, configuration, automation, security and testing of EventForce. It covers:

- Business overview and objectives.
- Phase-by-phase implementation: requirements, development, UI, testing, security.
- Data model and relationships (Event, Client, Vendor, Venue, Feedback, Event Vendor).
- Automation: Flow, Approval Process, validation rule and Apex logic.
- Security model: profiles, roles, permission sets, OWD and sharing rules.
- Test results and screenshots of the Salesforce features.

It serves as a blueprint for future developers and administrators, supporting consistent development, training and knowledge transfer.

7. Functional & Performance Testing

Screenshots were captured from Salesforce while testing event creation, venue assignment, the double-booking trigger, approval status changes, batch processing, scheduled reminders and dashboard analytics. All features behaved correctly.

Testing covered data accuracy, automation performance and user permissions. The main scenarios are listed below.

#	Area	Test scenario	Expected result
1	Validation rule	Save a Client with an invalid email such as "abc@"	Save is blocked with "Please Enter Valid Email Address"
2	Formula field	Create an event of each Event Type	Event Budget fills in according to the formula
3	Double booking	Create two events at the same venue on the same date	Error: "This Venue is already booked on this date."
4	Venue status	Set an event to Confirmed, then Canceled	Venue becomes Reserved, then Available
5	Approval process	Set status to Pending Cancellation and submit	Approval request and alert emails are sent
6	Approval – approve	Approver approves the request	Status = Canceled; owner notified
7	Approval – reject	Approver rejects the request	Status = Rejected; owner notified
8	Flow	Confirm an event 3+ days ahead with a client email	Reminder is sent three days before the event date
9	Batch Apex	Run the batch with past-dated events	Past events are marked Completed
10	Lookup filter	Pick a client on Feedback, then open the Event lookup	Only that client's events are listed
11	Security	Log in as each user type	Each user sees only permitted tabs, objects and records
12	Sharing rule	Vendor Manager views coordinator-owned events	Events are visible as Read Only

8. Advantages & Disadvantages

Advantages

- **Centralised data** – Event, Client, Vendor, Venue, Feedback and EventVendor records live in Salesforce.
- **Process automation** – Flows send reminders and run other steps, cutting manual work.
- **No double bookings** – the Apex trigger and VenueStatusHelper validate availability and prevent conflicts.
- **Controlled cancellations** – the Approval Process handles requests in a structured way.
- **Operational visibility** – reports and dashboards show schedules, vendor activity and feedback.
- **Secure access** – profiles, roles, permission sets, OWD and sharing rules control who sees what.
- **Scalable** – the platform can support growth and new requirements.
- **Better data quality** – validation rules and pre-checks during migration keep data consistent.

Disadvantages

- **Developer Edition only** – built for academic purposes and not connected to a production environment.
- **No production rollout** – migration to production was out of scope.
- **Extra tooling for real deployment** – sandboxes, change sets, Salesforce DX or CI/CD pipelines may be needed.
- **Configuration complexity** – Flows, Apex, approvals and security need Salesforce expertise.
- **Ongoing maintenance** – flows, Apex jobs, approvals, validation rules and sharing settings need regular monitoring.

9. Conclusion

EventForce shows how Salesforce can streamline and automate event planning and management. Events, Clients, Vendors, Venues, Feedback and EventVendor data now sit on one platform.

The implementation combines Lightning, Flows, validation rules, formula fields, approval processes, Apex triggers and classes, Batch and Scheduled Apex, security controls, reports and dashboards. Automation covers event reminders, cancellation approvals, venue availability, double-booking prevention and event completion.

The project also highlights how important data migration, testing, security, documentation and maintenance are in a Salesforce rollout. Although it was built and tested in a Developer Edition org without a production deployment, it follows real-world Salesforce practice and gives a solid base for future growth.

Key Learnings

- Designing a scalable data model, including junction objects for many-to-many links.
- Combining declarative tools (Flows, approvals, validation rules) with Apex for advanced logic.
- Writing bulk-safe Apex triggers and using Batch and Schedulable Apex for asynchronous work.
- Layering security with profiles, roles, permission sets, OWD and sharing rules.
- Turning operational data into decisions through reports and dashboards.
- Seeing how a Developer Org build maps to real deployment, maintenance and documentation practice.

10. Future Scope

EventForce can be extended to handle larger event operations and to improve automation, integration and usability.

- **Production deployment** – move the solution to a production org using proper deployment tools.
- **Wider event automation** – automate more planning and operational activities.
- **Richer vendor management** – add more advanced vendor tracking and performance measures.
- **Advanced analytics** – build more reports and dashboards on events, venues, vendors and feedback.
- **External integrations** – connect Salesforce with the outside systems that event operations rely on.
- **Mobile access** – make event management convenient on mobile devices.
- **Greater scale** – support larger numbers of events, clients, vendors, venues and users.
- **Continuous optimisation** – keep improving flows, Apex, validation rules and sharing settings as requirements change.

With these enhancements, EventForce can grow into a fuller, enterprise-ready event management CRM.

11. Appendix

Ref	Topic	Contents
A	Custom objects	Event, Client, Vendor, Venue, EventVendor, Feedback
B	Automation	Flows, validation rules, approval process, Apex triggers and classes
C	Processing	Batch Apex and Scheduled Apex
D	Security	Profiles, roles, permission sets, OWD, sharing rules
E	Data migration	Salesforce Data Import Wizard
F	UI & analytics	Lightning App, reports and dashboards
G	Testing & documentation	Configuration, security and automation testing; project documentation

Thank You